ARTIFICIAL INTELLIGENCE

Unit-I: Introduction

Unit-I: Introduction

Introduction

- AI problems, Agents and Environments
- Structure of Agents, Problem Solving Agents

Basic Search Strategies

- Problem Spaces
- Uninformed Search (Breadth-First, Depth-First Search, Depth-first with Iterative Deepening),
- Heuristic Search (Hill Climbing, Generic Best-First, A*)

AI Role in Mechanical Engineering

- Machine learning enables **predictive monitoring,** with machine learning algorithms **forecasting** equipment **breakdowns** in manufacturing units before they occur and scheduling timely maintenance.
- Artificial intelligence (AI) is also being adopted for product **inspection** and **quality control**.
- According to Forbes, automated quality testing done with machine learning can increase detection rates by up to **90%**

AI Role in Mechanical Engineering

- CIM (**Computer Integrated Manufacturing** such as (analysis, CAD, CAM, etc)) needs transparent **communications** as different systems handle the same object using different representations, thus requiring translation.
- David (1987) discusses the requirements of an Intelligent CAD system, and introduces a system called **Ciao** (Computer Intelligently Assisted Design).
- AI Can assist in those scenarios.

AI Role in Mechanical Engineering

There are different areas where AI impacts the Mechanical Engineering process. The idea behind the work of AI remains the same. It performs activities without humans yet with increased tendency as compared to humans. It is prioritizing the automated part of work, where we feed the computer with data, and as per the command, the machine/ process continues its function.

AI Role in Mechanical Engineering

Whenever we start the process of building a component/product/flow, the first step of it would be of Mechanical Design.

A. I. can majorly impact Product Design Services when it comes to designing the concept, examining the product, and also during the manufacturing of the product.

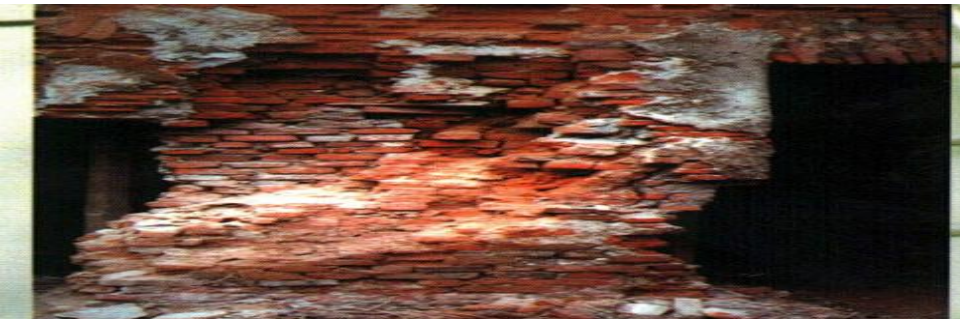
AI Role in Mechanical Engineering

Stress Elimination in 3D Structures

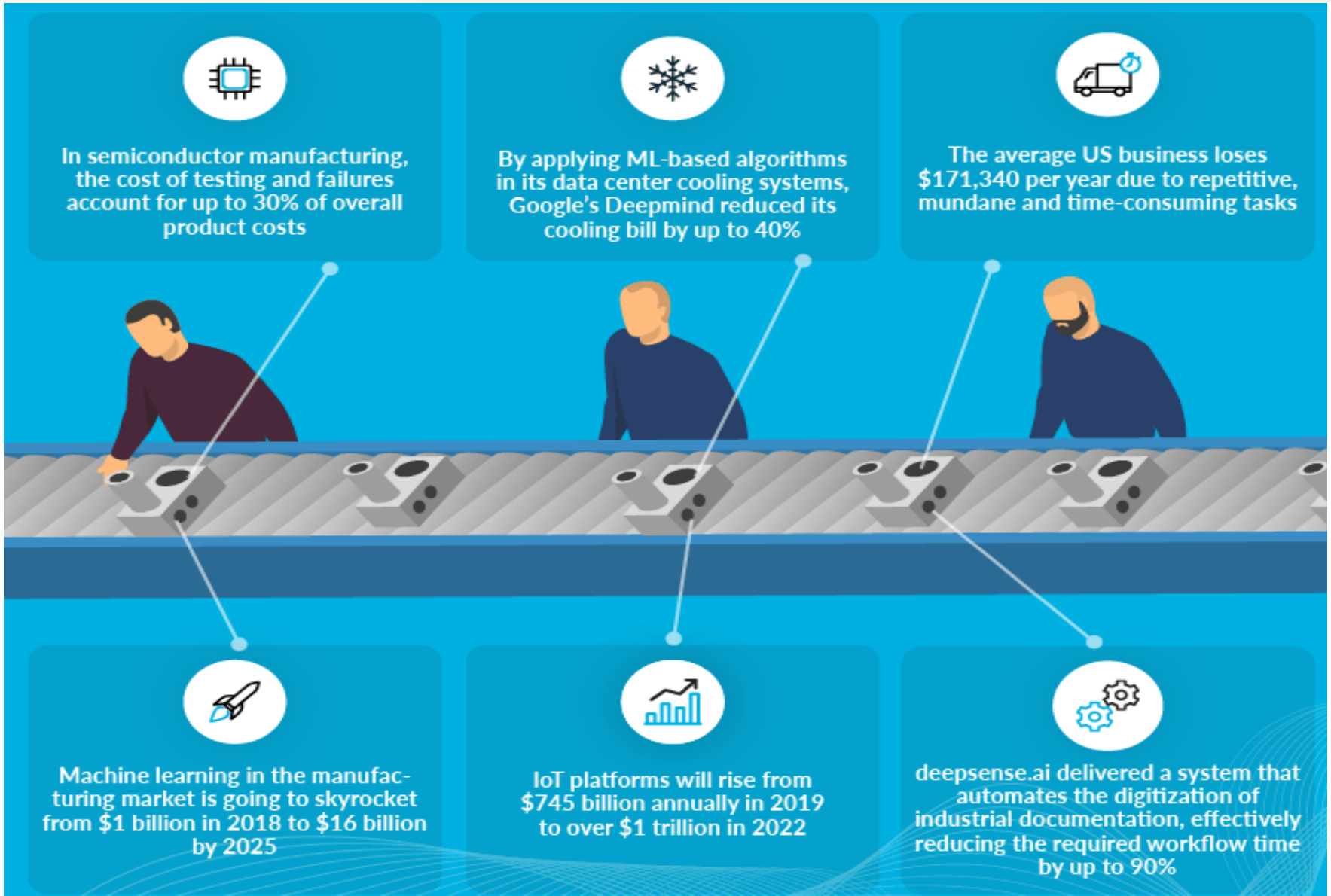
- Estimating the amount of stress while designing and manufacturing 3D Structures

Material Evaluation for Different Services

- Evaluating materials, its Strength-durability-quality and helping in a more exceptional manufacturing process



AI Role in Mechanical Engineering



Intelligence and Artificial Intelligence (AI)

- **Intelligence** is the ability to **acquire, understand** and **apply** the knowledge to achieve goals in the world

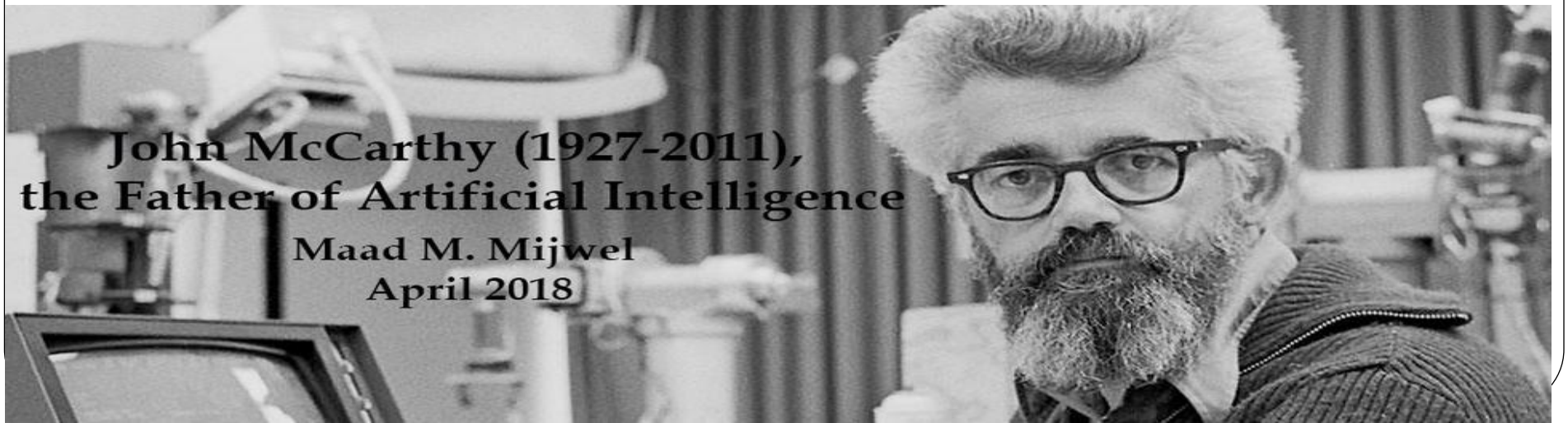
  Acquire → Understand → Apply → GOAL

- On the other hand, **Artificial intelligence** (AI) is the study of how to make **computers** do things which, at the moment, people do better

Artificial Intelligence (AI)

- Artificial intelligence (AI) is the **intelligence of machines** and the branch of **computer science** which aims to create **intelligent** machines.
- Artificial Intelligence is concerned with the **design** of intelligence in an **artificial device**.

The term was coined by McCarthy in 1956.



Artificial Intelligence (AI)

- Artificial Intelligence is concerned with the design of **intelligence** in an **artificial device**.

- There are two ideas in the definition.
  - 1. Intelligence
  - 2. artificial device

– A system with intelligence is **expected** to **behave** as intelligently as a human

– A system with intelligence is expected to behave in the **best** possible **manner**

Definition of AI

- Major AI textbooks define the field as

  "the study and design of intelligent agents,"

where an intelligent agent is a system that perceives its environment and takes actions which maximize its chances of success.

Thinking Humanly	Thinking Rationally
"The exciting new effort to make computers think . . . *machines with minds*, in the full and literal sense." (Haugeland, 1985) "[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning . . ." (Bellman, 1978)	"The study of mental faculties through the use of computational models." (Charniak and McDermott, 1985) "The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992)
Acting Humanly	**Acting Rationally**
"The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990) "The study of how to make computers do things at which, at the moment, people are better." (Rich and Knight, 1991)	"Computational Intelligence is the study of the design of intelligent agents." (Poole *et al.*, 1998) "AI . . . is concerned with intelligent behavior in artifacts." (Nilsson, 1998)

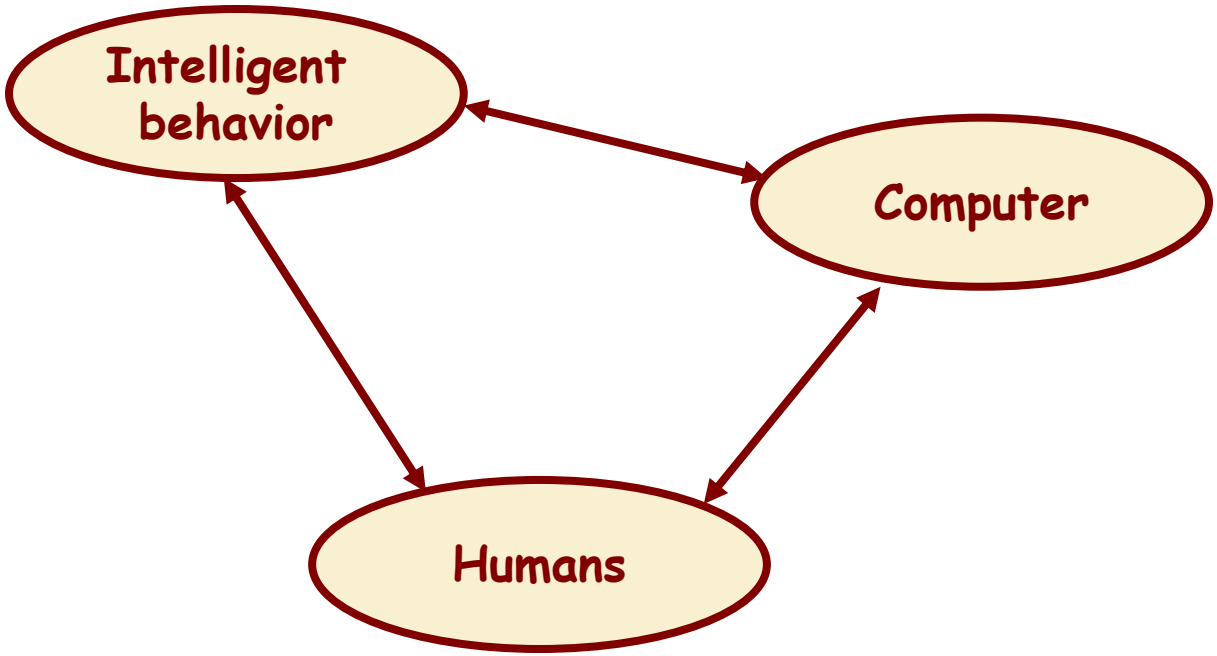
Figure 1.1 Some definitions of artificial intelligence, organized into four categories.

Artificial intelligence is about designing systems that are as intelligent as humans.

What is AI?

an attempt of

- AI is ~~the~~ reproduction of human reasoning and intelligent behavior by computational methods



What is AI?

Discipline that systematizes and automates reasoning processes to create machines that:

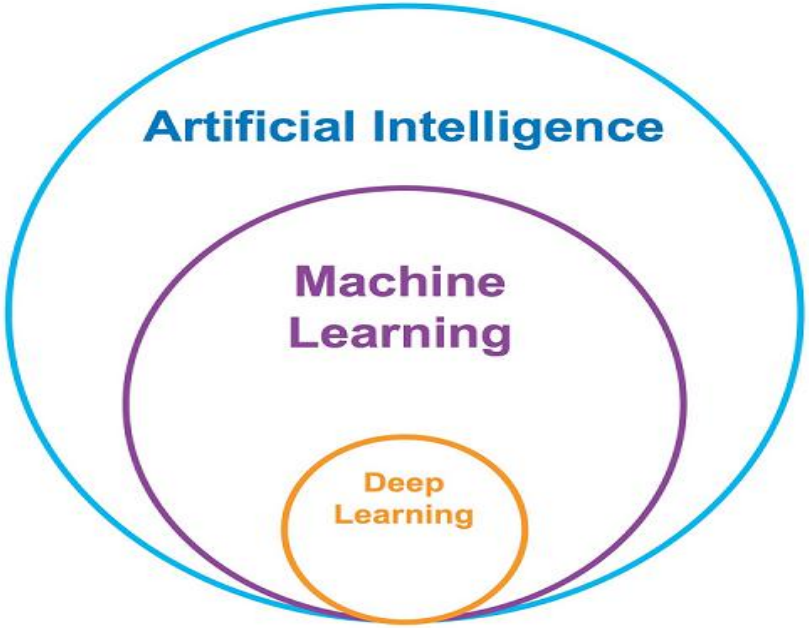
Act like humans	Act rationally
Think like humans	Think rationally

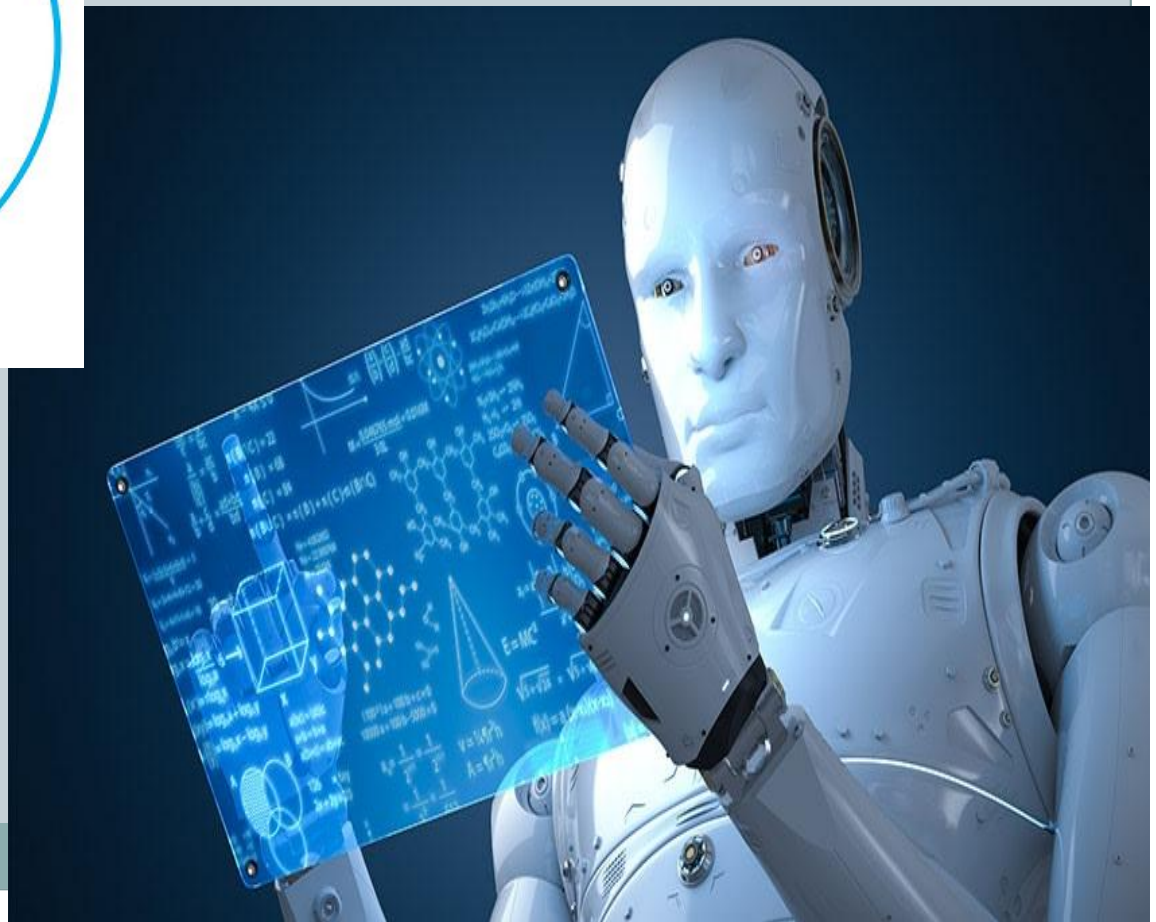
Act like humans	Act rationally
Think like humans	Think rationally

- The goal of AI is to create computer systems that perform tasks regarded as requiring **intelligence** when done by humans
- → AI Methodology: Take a task at which people are better, e.g.:
  - Prove a theorem
  - Play chess
  - Plan a surgical operation
  - Diagnose a disease
  - Navigate in a building

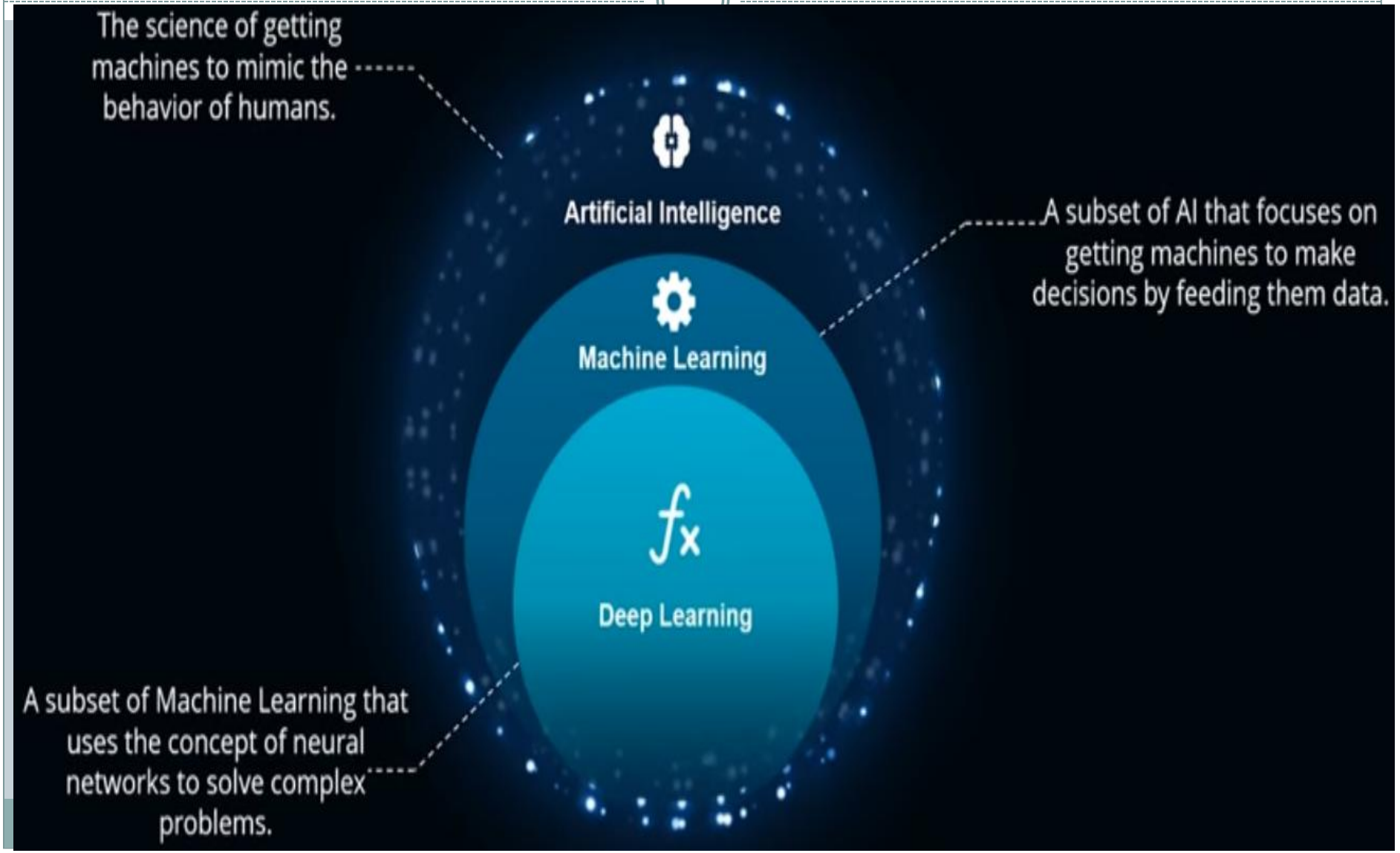
  and build a computer system that does it automatically
- But do we want to duplicate human imperfections?

AI, ML and Deep Learning





AI, ML and Deep Learning



NOTES

What is Artificial Intelligence ?

- making computers that think?
- the automation of activities we associate with human thinking, like decision making, learning ... ?
- the art of creating machines that perform functions that require intelligence when performed by people ?
- the study of mental faculties through the use of computational models ?

What is Artificial Intelligence ?

- the study of computations that make it possible to perceive, reason and act ?
- a field of study that seeks to explain and emulate intelligent behaviour in terms of computational processes ?
- a branch of computer science that is concerned with the automation of intelligent behaviour ?
- anything in Computing Science that we don't yet know how to do properly ? (!)

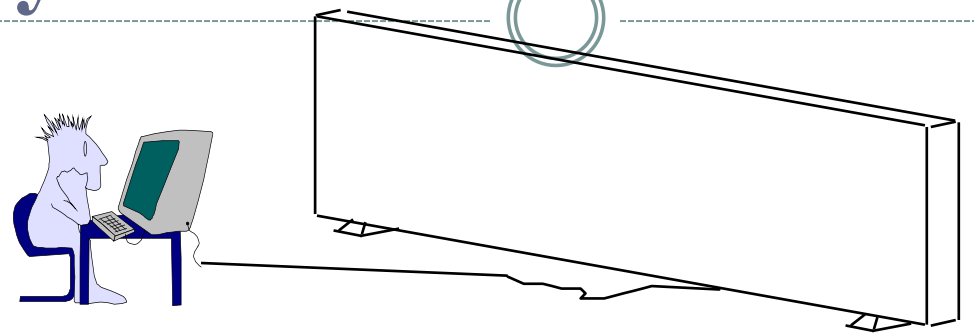
What is Artificial Intelligence ?

	HUMAN	RATIONAL
THOUGHT	Systems that think like humans	Systems that think rationally
BEHAVIOUR	Systems that act like humans	Systems that act rationally

Turing Test:
Systems that act like humans

- "The art of creating **machines** that perform **functions** that require **intelligence** when performed by **people**." (Kurzweil)
- "The study of how to make **computers** do things at which, at the moment, people are better." (Rich and Knight)

Systems that act like humans

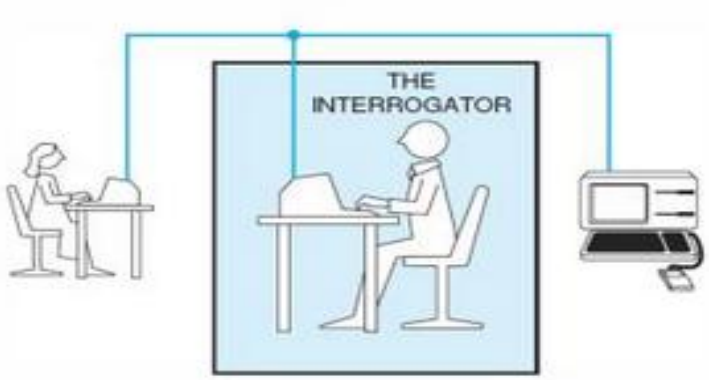


?

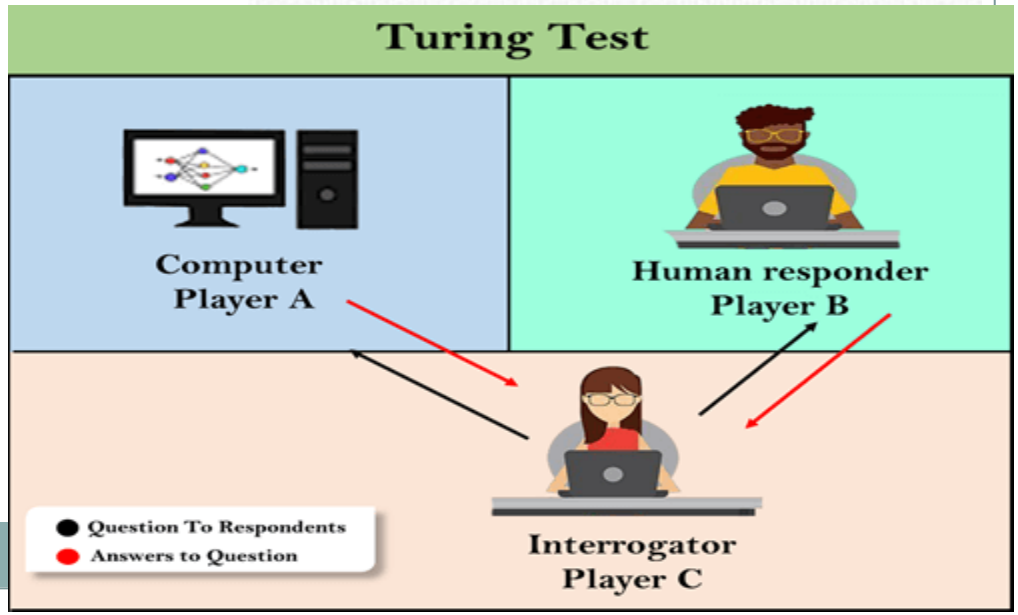
- You enter a room which has a computer terminal. You have a fixed period of time to type what you want into the terminal, and study the replies. At the other end of the line is either a **human** being or a computer system.
- If it is a computer system, and at the end of the period you cannot reliably determine whether it is a system or a human, then the system is **deemed** to be intelligent.

Turing Test

- Mathematician Alan Turing devised a test for defining artificial intelligence:
  - an interrogator poses questions to two entities, a human and a computer



- If the interrogator cannot tell which is the human and which is the computer, then the computer passes the Turing Test and should be considered intelligent
  - Turing Test – a test for machine intelligence



The Total Turing Test

- Includes two more issues:
  - *Computer vision*
    - to perceive objects (seeing)
  - *Robotics*
    - to move objects (acting)

Systems that act like humans

- The Turing Test approach
  - a human questioner cannot tell if
    - there is a computer or a human answering his question, via teletype (remote communication)
  - The computer must behave intelligently
- Intelligent behavior
  - to achieve human-level performance in all cognitive tasks

Systems that act like humans

- These cognitive tasks include:
  - ***Natural language processing***
    - for communication with human
  - ***Knowledge representation***
    - to store information effectively & efficiently
  - ***Automated reasoning***
    - to retrieve & answer questions using the stored information
  - ***Machine learning***
    - to adapt to new circumstances

AI problems/What can AI do today?

- While studying the **typical** range of **tasks** that we might expect an "**intelligent entity**" to perform, we need to consider both "common-place/**Mundane**/routine tasks as well as **expert** tasks.

- Mundane tasks: (daily use tasks)
  - Planning **route**, activity
  - Recognizing **people**, **objects** (vision, speech)
  - **Communicating** (through natural language)
  - **Navigating** around obstacles on the street.

AI problems (cont..)

- **Expert Tasks**
  - Medical diagnosis
  - Mathematical problem solving
  - Playing games like chess

- Easy /Hard Problems: It has been easier to mechanize many of the high level tasks (expert). We usually associate with the intelligence in people, in history we don't have same amount of success with mundane tasks.

AI Problems

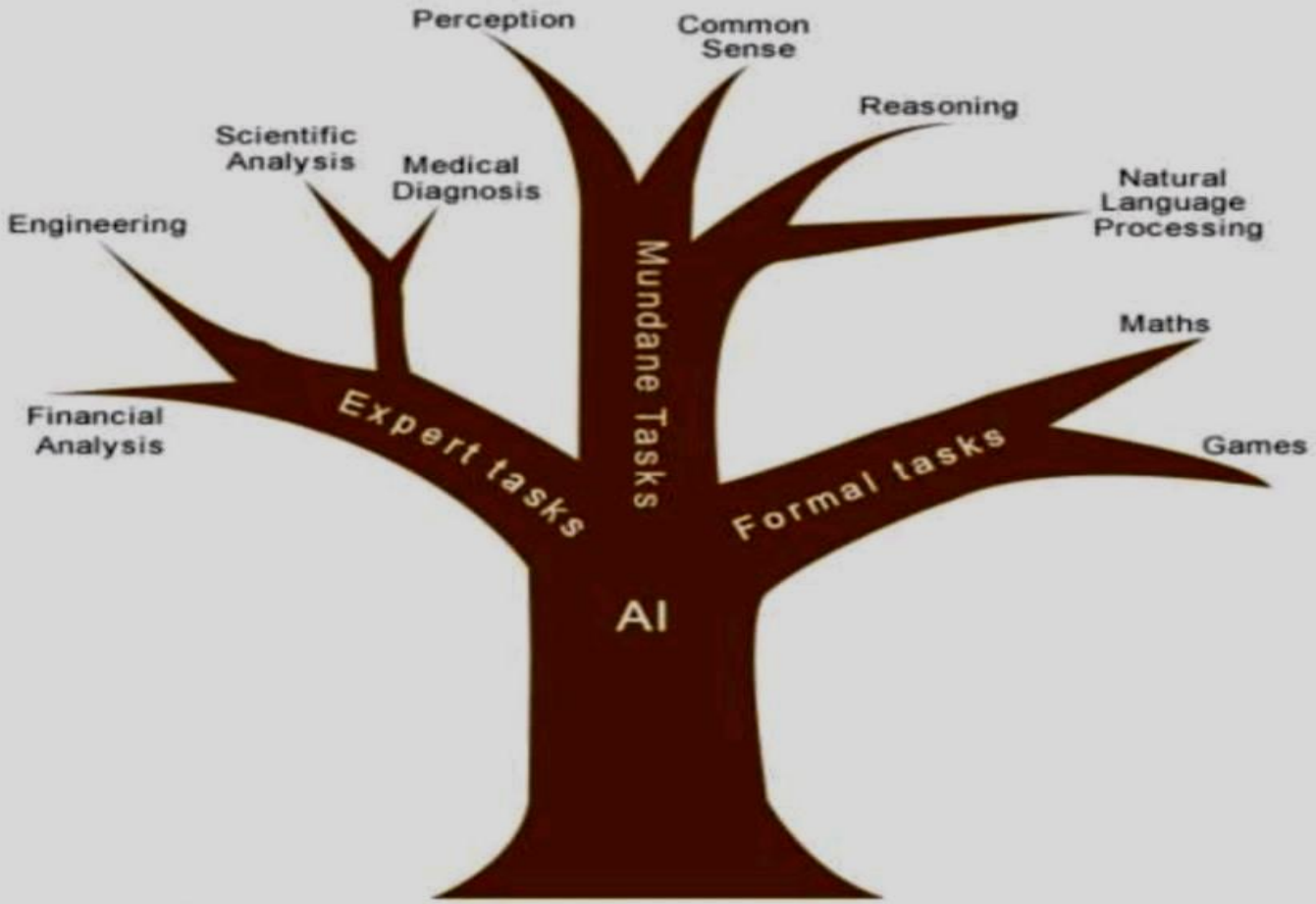
Task Domains of AI

- Mundane Tasks:
  - Perception
    - Vision
    - Speech
  - Natural Languages
    - Understanding
    - Generation
    - Translation
  - Common sense reasoning
  - Robot Control
- Formal Tasks
  - Games : chess, checkers etc
  - Mathematics: Geometry, logic,Proving properties of programs
- Expert Tasks:
  - Engineering (Design, Fault finding, Manufacturing planning)
  - Scientific Analysis
  - Medical Diagnosis
  - Financial Analysis

AI Problems

Task Classification of AI

The domain of AI is classified into *Formal* tasks, *Mundane* tasks, and *Expert* tasks.



AI problems (cont..)

- AI Systems can easily do
  - **Symbolic integration (lists, hierarchies, or networks)**
  - **Proving theorems**
  - **Playing chess**
  - **Medical Diagnosis**
- It has been very **hard** to **mechanize tasks** that lot of animals do
  - Walking around without running into things
  - Catching Prey and avoiding predators
  - Interpreting complex sensory information
  - Modeling the internal states of other animals from their behavior.

AI problems (cont..)

General Tasks	Formal Tasks	Expert Tasks
Perception -	**Games** 	**Engineering**
Vision and Speech	Chess, Backgammon, Checkers	Design, Fault finding, Manufacturing Planning
Natural Language - Processing	**Mathematics**	**Scientific Analysis**
Understanding, Generation, Translation	Geometry, Logic, Integral Calculus, Proving Properties of Program	**Medical Diagnosis**
Common Sense Reasoning and Robot Control		**Financial Analysis**

AI problems (cont..)

- **Humans** have learned mundane (ordinary) tasks since their **birth**.
- They learn by **perception, speaking**, using **language**, and **locomotives**.

- They learn Formal Tasks and Expert Tasks **later**, in that order.
- For humans, the mundane tasks are easiest to learn. The same was considered true before trying to implement mundane tasks in machines.

AI problems (cont..)

- Earlier, all work of AI was concentrated in the **mundane** task domain.
- Later, it turned out that the **machine** requires more **knowledge**, **complex** knowledge representation, and complicated algorithms for handling mundane tasks.
- This is the reason why AI work is more prospering in the **Expert Tasks** domain now, as the expert task domain needs expert knowledge without common sense, which can be easier to represent and handle.

FYI on AI

Limits of AI Today

- Today's successful AI systems operate in well-defined domains and employ narrow, specialized knowledge.
- Common sense knowledge is needed to function in complex, open-ended worlds. Such a system also needs to understand unconstrained natural language. However these capabilities are not yet fully present in today's intelligent systems.

What can AI systems do

Today's AI systems have been able to achieve limited success in some of these tasks.

- In Computer vision, the systems are capable of face recognition
- In Robotics, we have been able to make vehicles that are mostly autonomous.
- In Natural language processing, we have systems that are capable of simple machine translation.
- Today's Expert systems can carry out medical diagnosis in a narrow domain
- Speech understanding systems are capable of recognizing several thousand words continuous speech

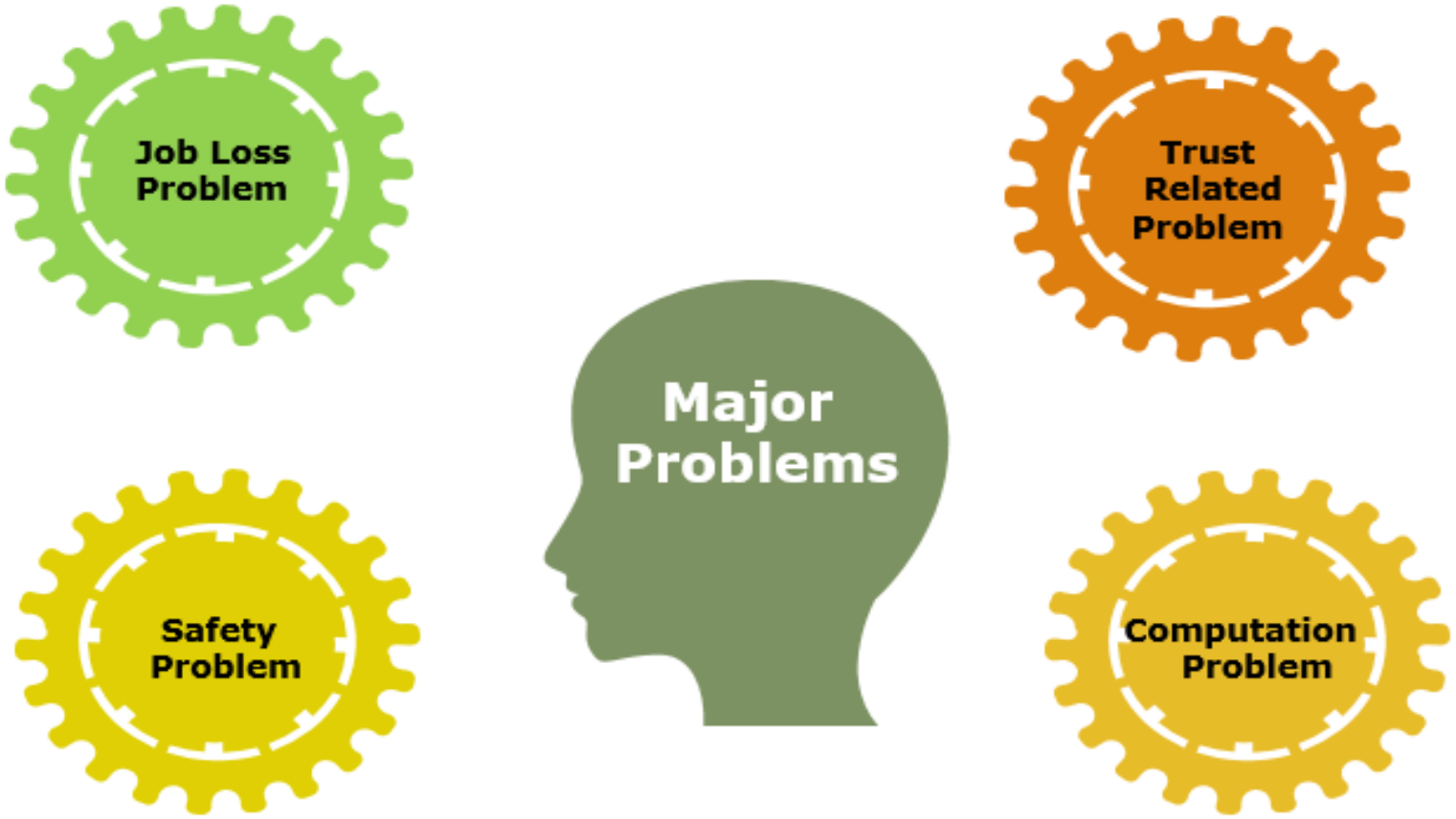
FYI on AI

- Planning and scheduling systems had been employed in scheduling experiments with the Hubble Telescope.
- The Learning systems are capable of doing text categorization into about a 1000 topics
- In Games, AI systems can play at the Grand Master level in chess (world champion), checkers, etc.

What can AI systems NOT do yet?

- Understand natural language robustly (e.g., read and understand articles in a newspaper)
- Surf the web
- Interpret an arbitrary visual scene
- Learn a natural language
- Construct plans in dynamic real-time domains
- Exhibit true autonomy and intelligence

Disadvantage with AI



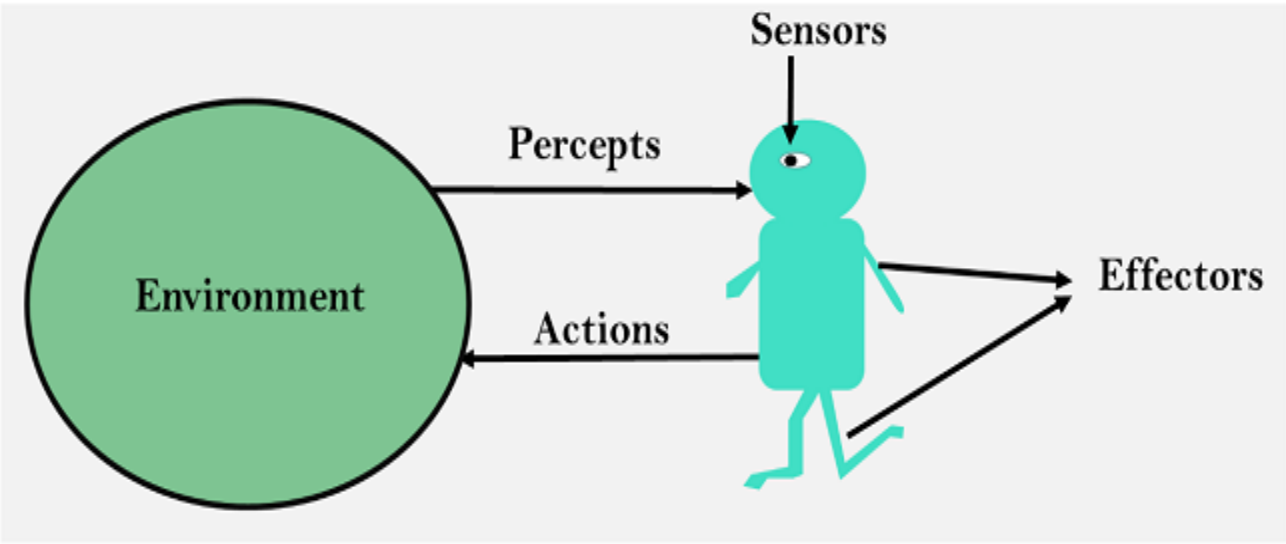
FYI: Today's AI Applications

- AI to analyze satellite images to identify which areas have the highest poverty levels
- New Artificial Intelligence now predicts the time it takes for a crop like a tomato to be ripe and ready for picking thus increasing the efficiency of farming.
- AI researchers have created many tools in AI laboratories to solve the most difficult problems in computer science. Thus include computer mouse, GUIs, Automatic storage management..etc.
- Computer-aided interpretation of medical images : A typical application is detection of the tumor.
- Artificial Intelligence have been combined with many sensor technologies which enables many applications such as at home water quality monitoring.
- Chat bots in Food order Apps.

Agents & Environments in AI

- An AI system can be defined as the study of the **rational agent** and its environment.
- The **agents** sense the **environment** through sensors and **act** on their environment through **actuators**.
- An AI agent can have mental properties such as knowledge, belief, intention, etc.

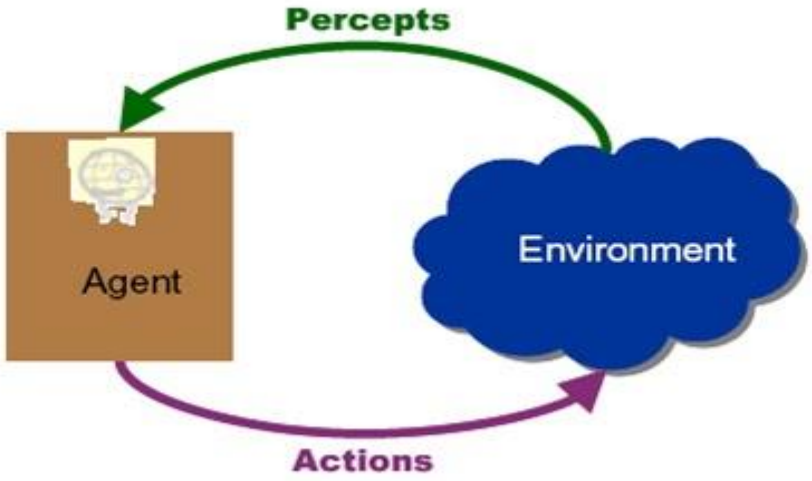
No Emotions and only Math based



Agents & Environments in AI

- An AI system can be defined as the study of the **rational agent** and its **environment**.
- The **agents** sense the **environment** through sensors and **act** on their environment through actuators.
- An AI agent can have **mental properties** such as knowledge, belief, intention, etc.

No Emotions and only
Math based

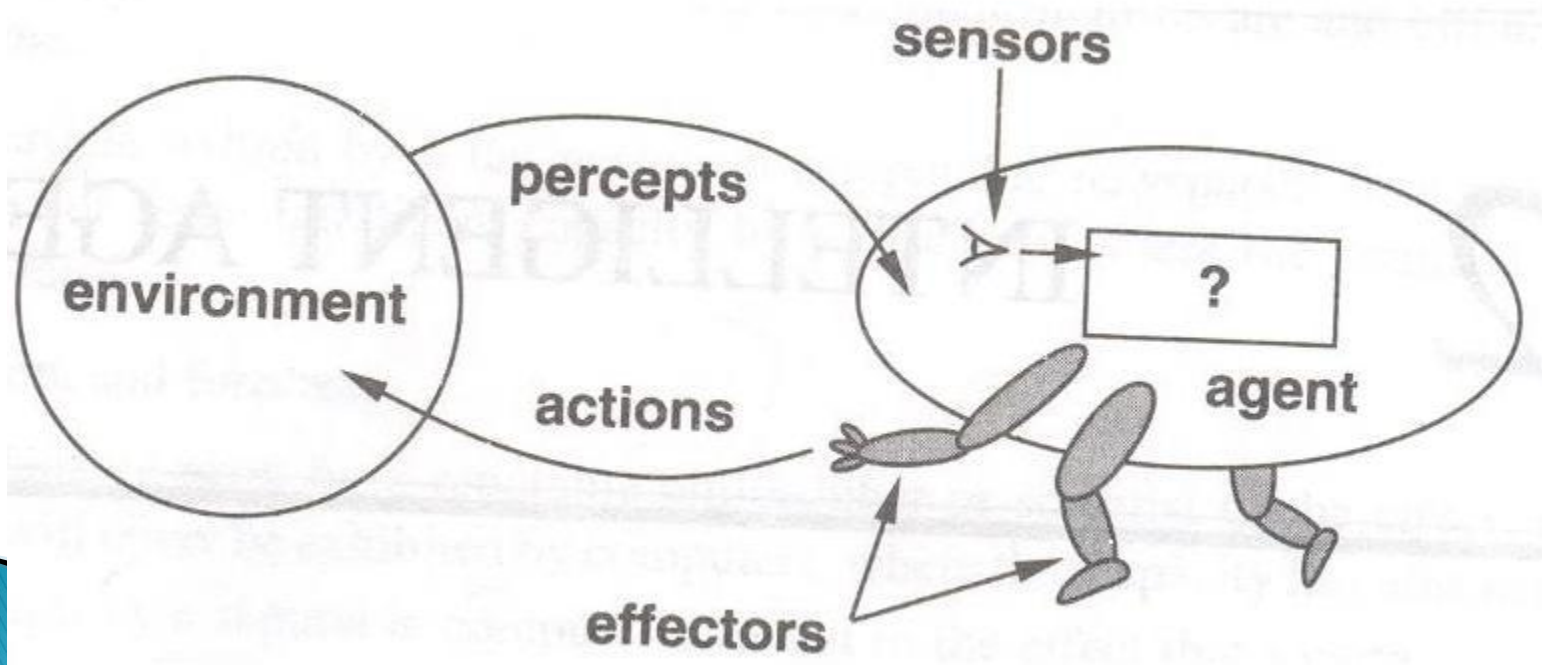


A rational agent
- is said to perform the right things
- which has clear preference, models uncertainty, and
acts in a way to maximize its performance measure with all possible actions

Agents & Environments

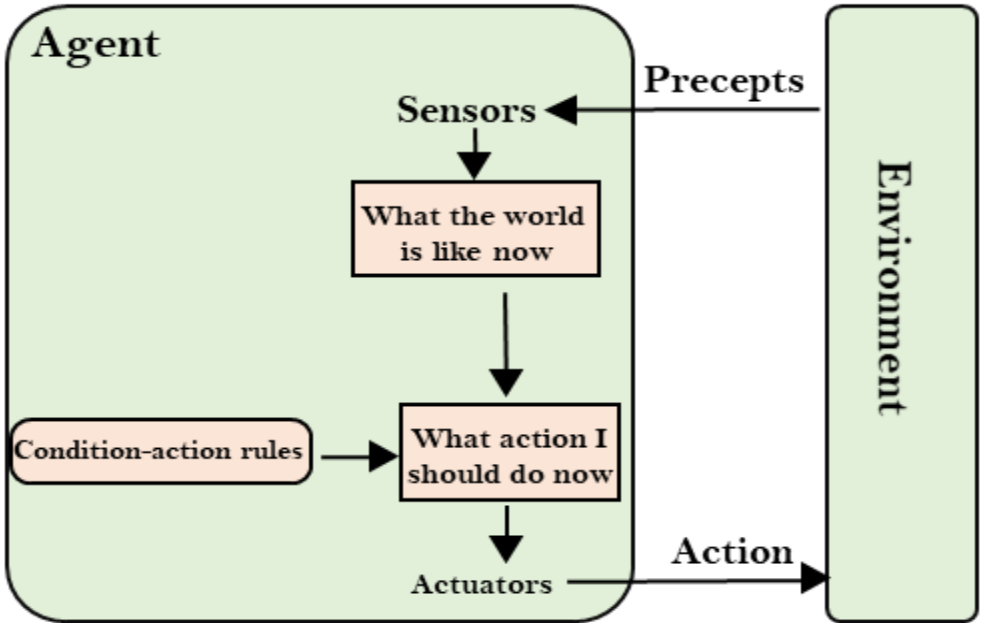
An agent can be anything that perceive its environment through sensors and act upon that environment through actuators.

An Agent runs in the cycle of **perceiving**, **thinking**, and **acting**.



FYI: Definitions

- **Agent:** An **agent** is anything that can perceive its environment through **sensors** and acts upon that environment through **effectors**.
- **Environment:** An **environment** is the surroundings or conditions in which an agents lives or operates.
- Action: An **action** is operation involving the effectors.(**Actuator).**



Working of Agent on Environment

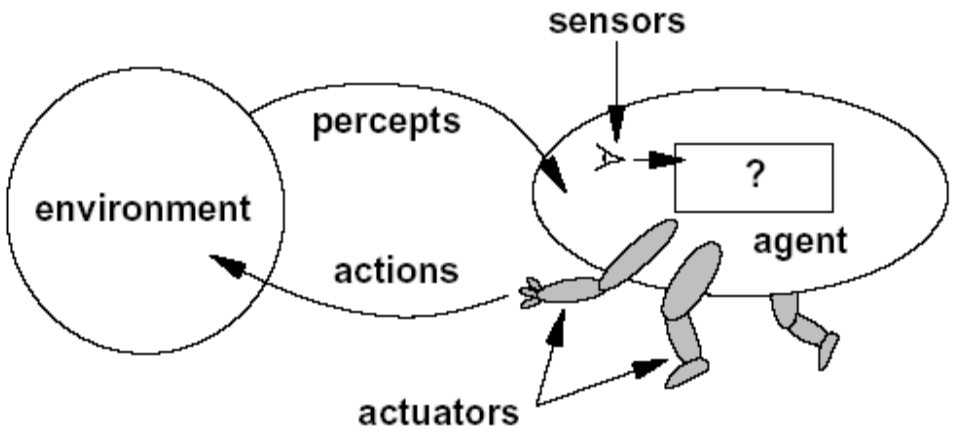
- An agent **perceives** its environment through **sensors**.
- The complete set of **inputs** at a given time is called a **percept**.
- The **current** percept, or a sequence of percepts can influence the **actions** of an agent.
- The agent can **change** the environment through actuators or effectors.
- An operation involving an effector is called an **action**.
- Actions can be **grouped** into action **sequences**.
- The agent can have **goals** which it tries to achieve.
- Thus, an agent can be looked upon as a system that implements a mapping from percept sequences to actions.

What is AI Agent?

- An agent can be anything that perceive its environment through **sensors** and act upon that environment through **actuators**.
- An Agent runs in the cycle of **perceiving**, **thinking**, and **acting**. An agent can be:
- **Human-Agent:** A human agent has eyes, ears, and other organs which work for sensors and hand, legs, vocal tract work for actuators.
- **Robotic Agent:** A robotic agent can have cameras, infrared range finder, Natural Lang. processor (NLP) for sensors and various motors for actuators.
- **Software Agent:** Software agent can have keystrokes, file contents as sensory input and act on those inputs and display output on the screen.

In other words….Agents

- A human agent has :
  - Sensors: eyes, ears, and other organs.
  - Actuator: hands, legs, mouth, and other body part.
- A robotic agent might have:
  - Sensors: Cameras,..
  - Actuator: motors
- A Software Agent
  - Sensors: ?
  - Actuator: ?
- The agent function



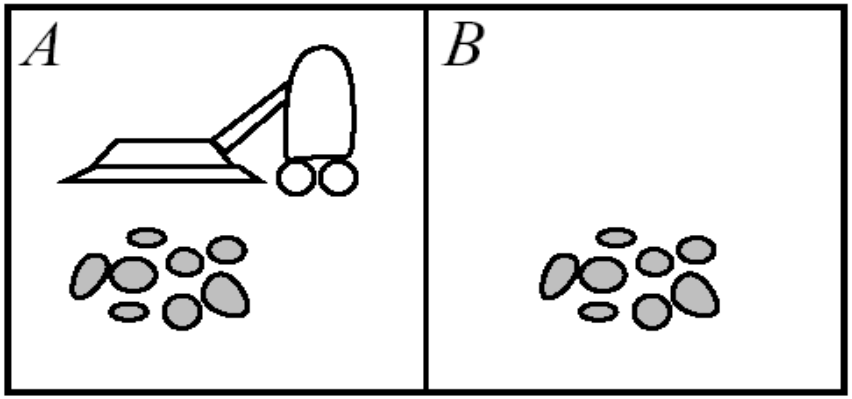
maps from percepts histories to actions

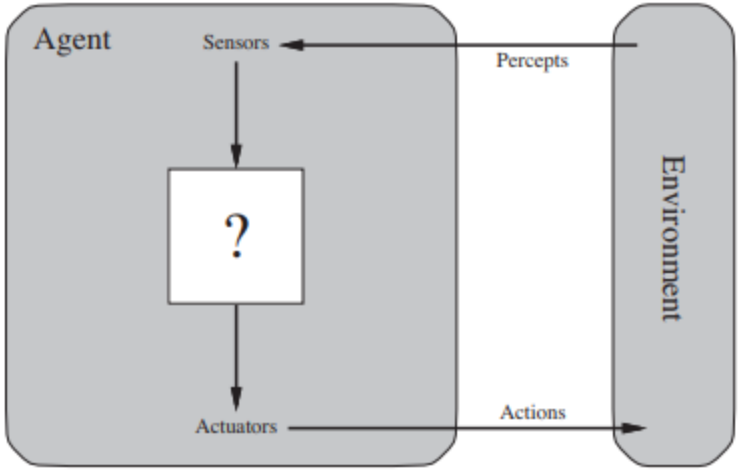
F: P→ A

Example: Vacuum Cleaner

- There are two locations: squares A and B.
- The vacuum agent **perceives** which square it is in and whether there is **dirt** in the square.
- It can choose to **move** left, move right, suck up the dirt, or do nothing.
- One very simple agent function is the following: if the current **square** is **dirty**, then **suck**, otherwise **move** to the other square.
- A partial tabulation of this agent function is shown in below Figure

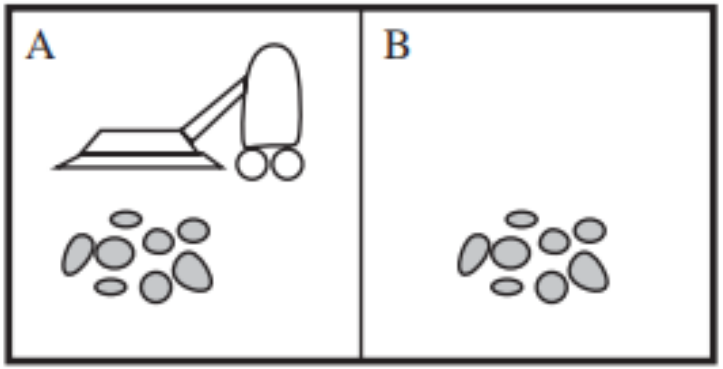
Example: Vacuum Cleaner





- **Agent**: vacuum cleaner
- **Environment**: Front room
- **Percept**: See dirt (Eye of the vacuum cleaner)
- **Action**: Suck the dirt (Attached components of vacuum cleaner
- **Change in the Environment**: Front room is clean

A Vacuum cleaner world with just two locations

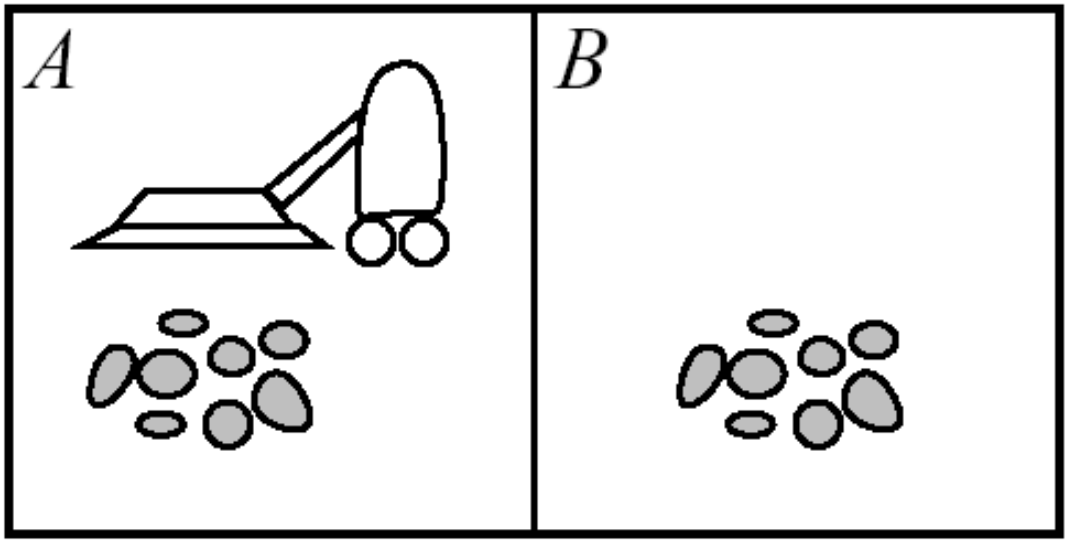


Percept sequence	Action
$[A, Clean]$	$Right$
$[A, Dirty]$	$Suck$
$[B, Clean]$	$Left$
$[B, Dirty]$	$Suck$
$[A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Dirty]$	$Suck$
⋮	⋮
$[A, Clean], [A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Clean], [A, Dirty]$	$Suck$
⋮	⋮

Example: Vacuum Cleaner Agent

- Agent: **robot** vacuum cleaner
- Environment: floors of your apartment
- Sensors:
  - **dirt sensor:** detects when floor in front of robot is dirty
  - **bump sensor:** detects when it has bumped into something
  - **power sensor:** measures amount of power in battery
  - **bag sensor:** amount of space remaining in dirt bag
- Effectors:
  - motorized wheels
  - suction motor
  - plug into wall? empty dirt bag?
- Percepts: "Floor is dirty"
- Actions: "Forward, 0.5 ft/sec"

Vacuum Cleaner Agent



Percepts: location and contents, e.g., $[A, Dirty]$

Actions: $Left$, $Right$, $Suck$, $NoOp$

Vacuum Cleaner Agent

Percept sequence	Action
$[A, Clean]$	$Right$
$[A, Dirty]$	$Suck$
$[B, Clean]$	$Left$
$[B, Dirty]$	$Suck$
$[A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Dirty]$	$Suck$
⋮	⋮

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action

    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left
```

Structure of AI Agents

- The job of AI is to design an **agent program** that implements the **agent function**—
  (ie., the mapping from percepts to→ actions)

- We assume this program will run on some sort of **ARCHITECTURE** computing device with physical **sensors** and **actuators**—we call this the **architecture**:

agent = architecture + program

In other Words… Structure of AI Agents

- The job of AI is to design an agent that implements the agent function.
- The agent function maps from percept histories to actions:

$$[f: \mathcal{P}^* \rightarrow \mathcal{A}]$$

- The agent program runs on the physical architecture (sensors & actuators) to produce f

agent = architecture + program

Structure of AI Agents

Following are the main three terms involved in the structure of an AI agent:

- **Architecture:** Architecture is **machinery** that an AI **agent** executes on.
- **Agent Function:** Agent **function** is used to **map** a **percept** (P*) to an **action** (A).

  $$f : P^* \rightarrow A$$
- **Agent program:** Agent **program** is an **implementation** of agent **function**.
- An agent program executes on the **physical architecture** to produce function f.

Structure of AI Agents

- Obviously, the program we choose has to be one that is appropriate for the architecture.
- If the program is going to recommend actions like *Walk*, *the architecture had better have legs*.
- *The* architecture might be just an ordinary PC, or it might be a robotic car with several onboard computers, cameras, and other sensors.
- In general, the architecture makes the **percepts** from the **sensors** available to the program, runs the program, and feeds the program's action choices to the **actuators** as they are generated.

Types of AI Agents

Types of AI Agent	Agent Working Process
State-based Agent or **model-based reflex agent**	✓information comes from **sensors - percepts** ✓based on this, the agent changes the current **state of the world** ✓based on **state of the world and knowledge (memory), it triggers actions through the effectors**
Goal-based Agent	**See Next Slide**
Utility-based Agent	Utility based agents provides a more general agent framework. In case that the agent has multiple goals, this framework can accommodate different preferences for the different goals. Such systems are characterized by a utility function that maps a state or a sequence of states to a real valued utility. The agent acts so as to maximize expected utility
Simple reflex agents	Select actions on the basis of current percept only **Condition-action rule:** if car-in-front-is-braking then initiate-braking
Learning Agent	Learning allows an agent to operate in initially **unknown** environments. The learning element modifies the performance element. Learning is required for true autonomy.

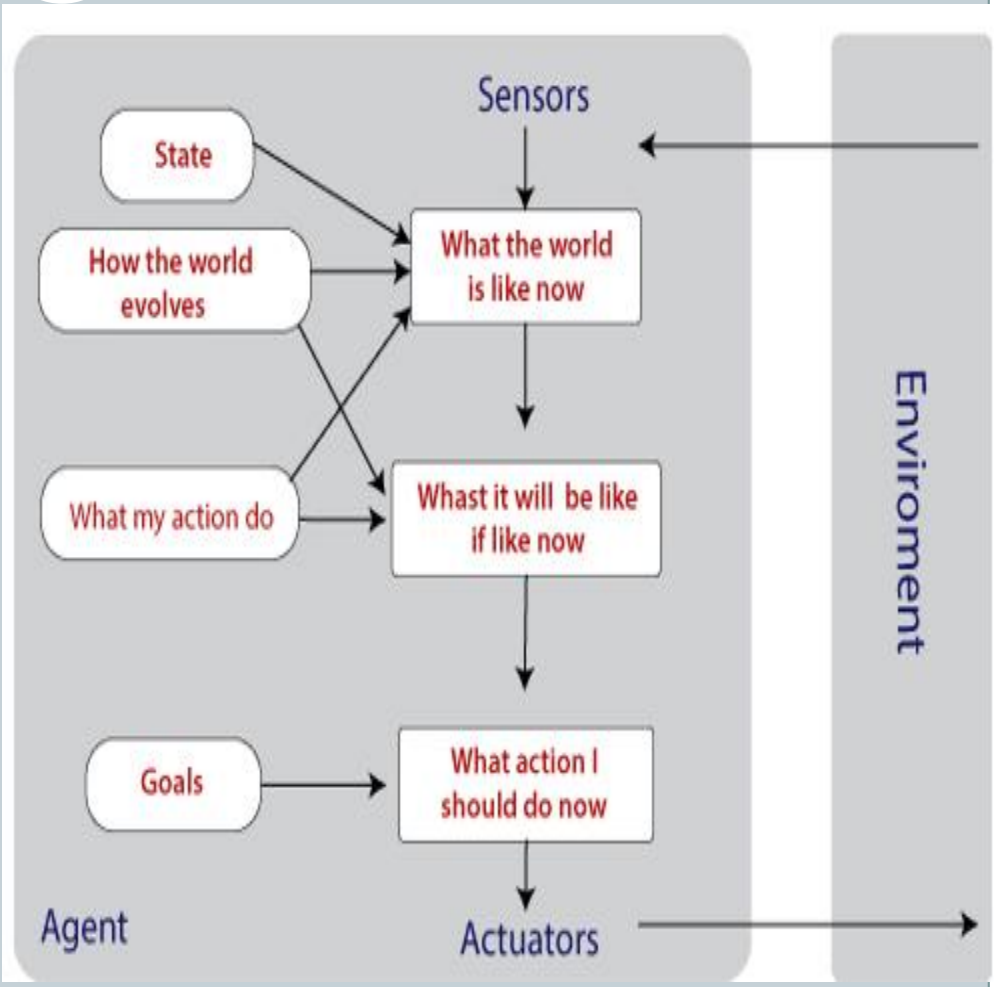
Goal-Based Agent

- State-based Agent or **model-based reflex agent** which base their actions on a **direct mapping** from **states to actions**.
- Such agents cannot operate well in environments for which this mapping would be too large to store and would take too long to learn.

- Goal-based agents, on the other hand, consider **future actions** and the **desirability** of their **outcomes**.

Goal-Based Agent

- In some cases, just **knowing** the information is not sufficient.
- The intent is to **reduce** distance from the **goal**, i.e., desired situations.
- Knowledge about the **goal** is an important **factor**.
- They are used to expand the capabilities **based** on the **goal** information.



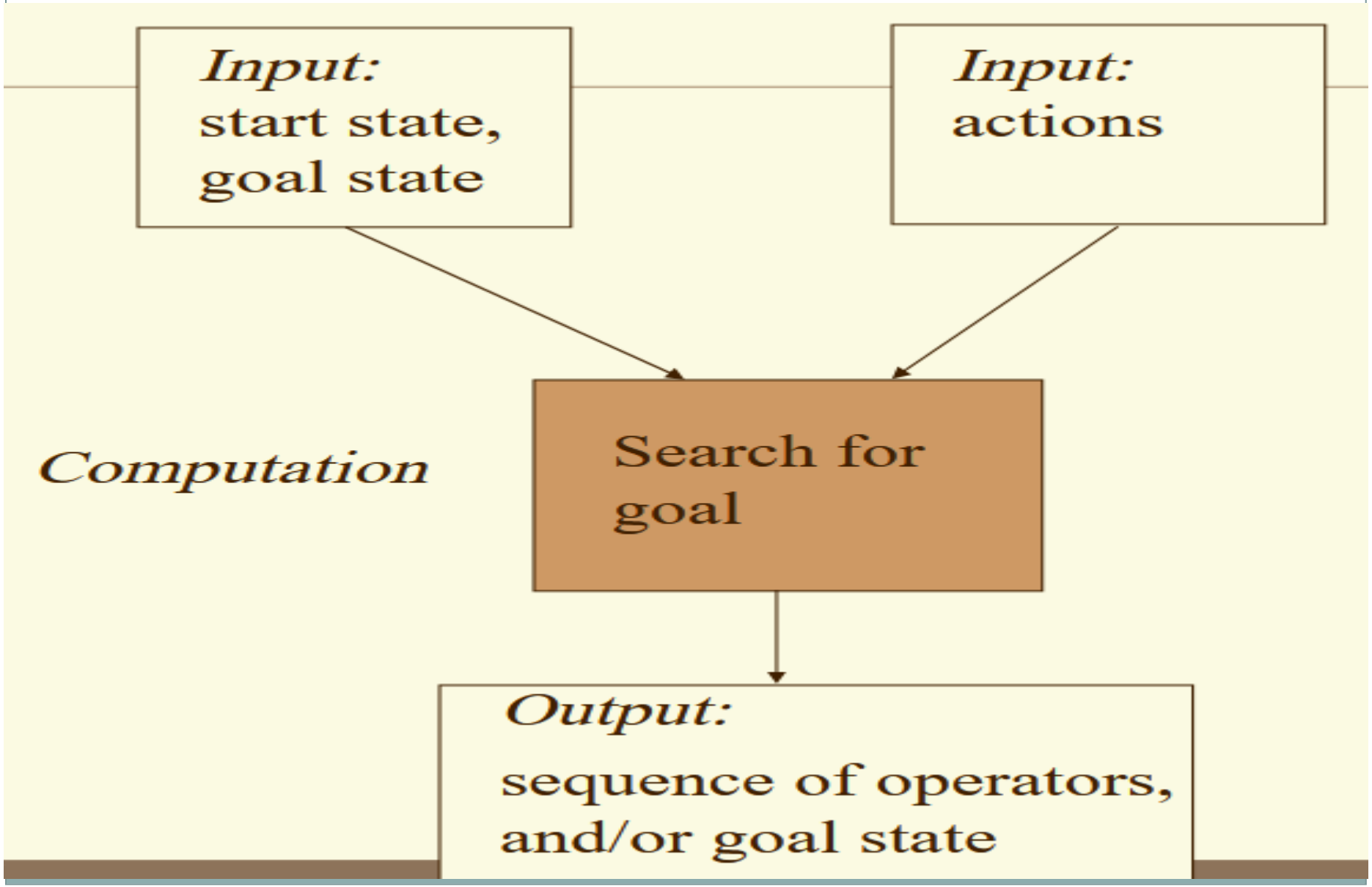
Goal-Based Agent

- The **goal based** agent has **some goal** which forms a **basis** of its **actions**. Such agents work as follows:
  - information comes from **sensors - percepts**
  - changes the agents current **state of the world**
  - based on **state of the world** and **knowledge (memory)** and **goals/intentions,** it chooses **actions** and does them through the **effectors.**
- Goal formulation based on the **current situation** is a way of solving many problems and **search** is a universal problem solving mechanism in AI.
- The **sequence** of steps required to solve a problem is not known a **priori** (from the former) and must be determined by a systematic exploration of the alternatives.

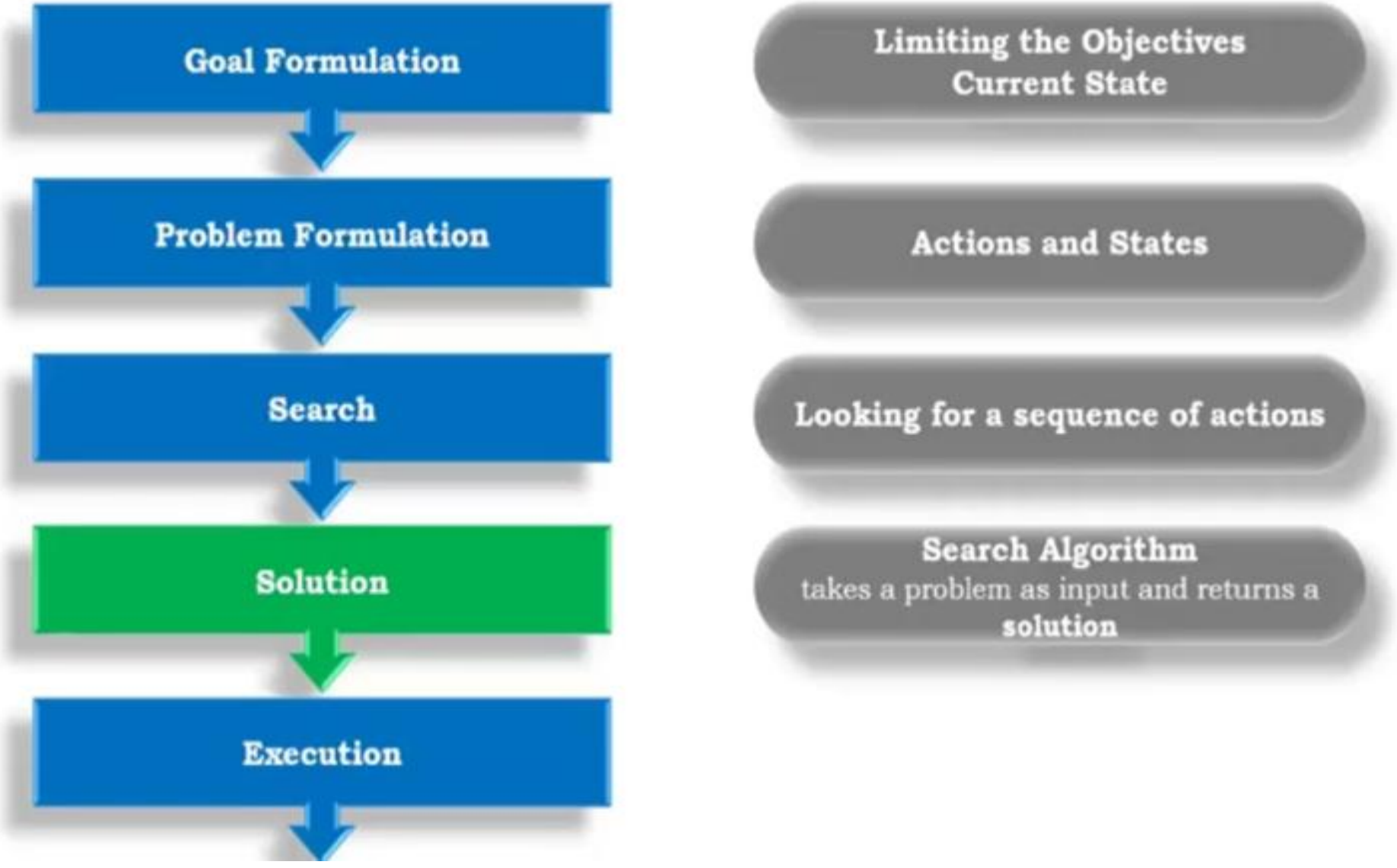
Problem-Solving Agents

- In Artificial Intelligence, **Search techniques** are universal problem-solving methods.
- **Rational agents** (one that does the right thing) or **Problem-solving agents** in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best **result**.
- Problem-solving agents are the goal-based agents and use atomic representation (each state of the world is a black box that has no internal structure).

Problem-Solving Agent



Problem-Solving Agent



Problem Solving Agent

- A Problem Solving has a GOAL
- Then the **Agent** Calls a **Search** **procedure** to solve it
- Problem Solving is a kind of **Goal-based Agent** that decides **What to do** by finding the **sequences** of **Actions** that lead to desirable states
- An Intelligent Agent case, a Knowledge Base (**KB**) corresponds to the **environment operators** correspond to **sensors** and the search techniques are the **Actuators**

Problem-Solving Agent

Steps Performed by Problem Solving Agent	Explanation
Goal Formulation	It organizes the **steps/sequence** required to **formulate** one **goal** out of multiple goals as well as actions to achieve that goal. Goal formulation is based on the **current situation** and the agent's performance measure.
Problem Formulation	It is the most important step of problem-solving which decides what **actions** should be taken to **achieve** the formulated goal.
Search	It identifies all the best possible sequence of actions to reach the goal state from the current state. It takes a problem as an input and returns solution as its output.
Solution	It finds the best algorithm out of various algorithms, which may be proven as the best optimal solution.
Execution	It executes the best optimal solution from the searching algorithms to reach the goal state from the current state.

Problem Formulation

This decides what actions should be taken to achieve the formulated goal.

There are following **five** components involved in problem formulation:

1. **Initial State:** It is the **starting state** or initial step of the agent towards its goal.
2. **Actions:** It is the description of the **possible actions** available to the agent.
3. **Transition Model:** It describes what each **action does**.
4. **Goal Test:** It determines if the given state is a goal state.
5. **Path cost:** It assigns a **numeric cost** to each path that follows the goal. The problem-solving agent selects a **cost function**, which reflects its performance measure.

Remember, **an optimal solution has the lowest path cost among all the solutions.**

Example: 8 puzzle

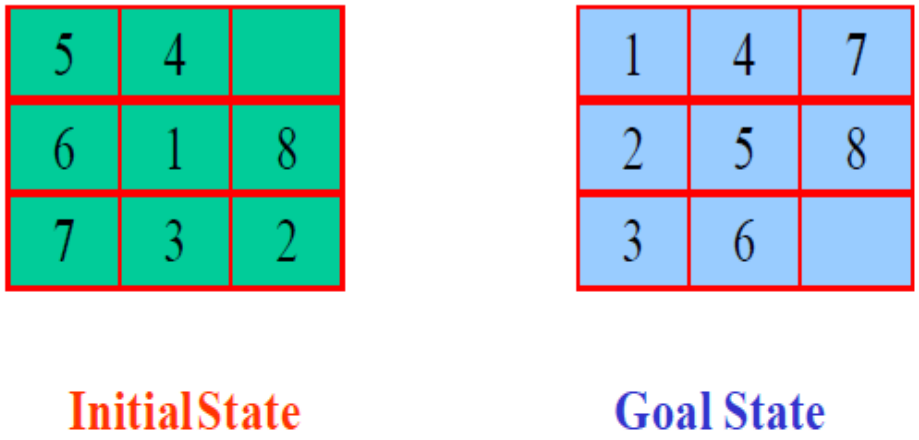
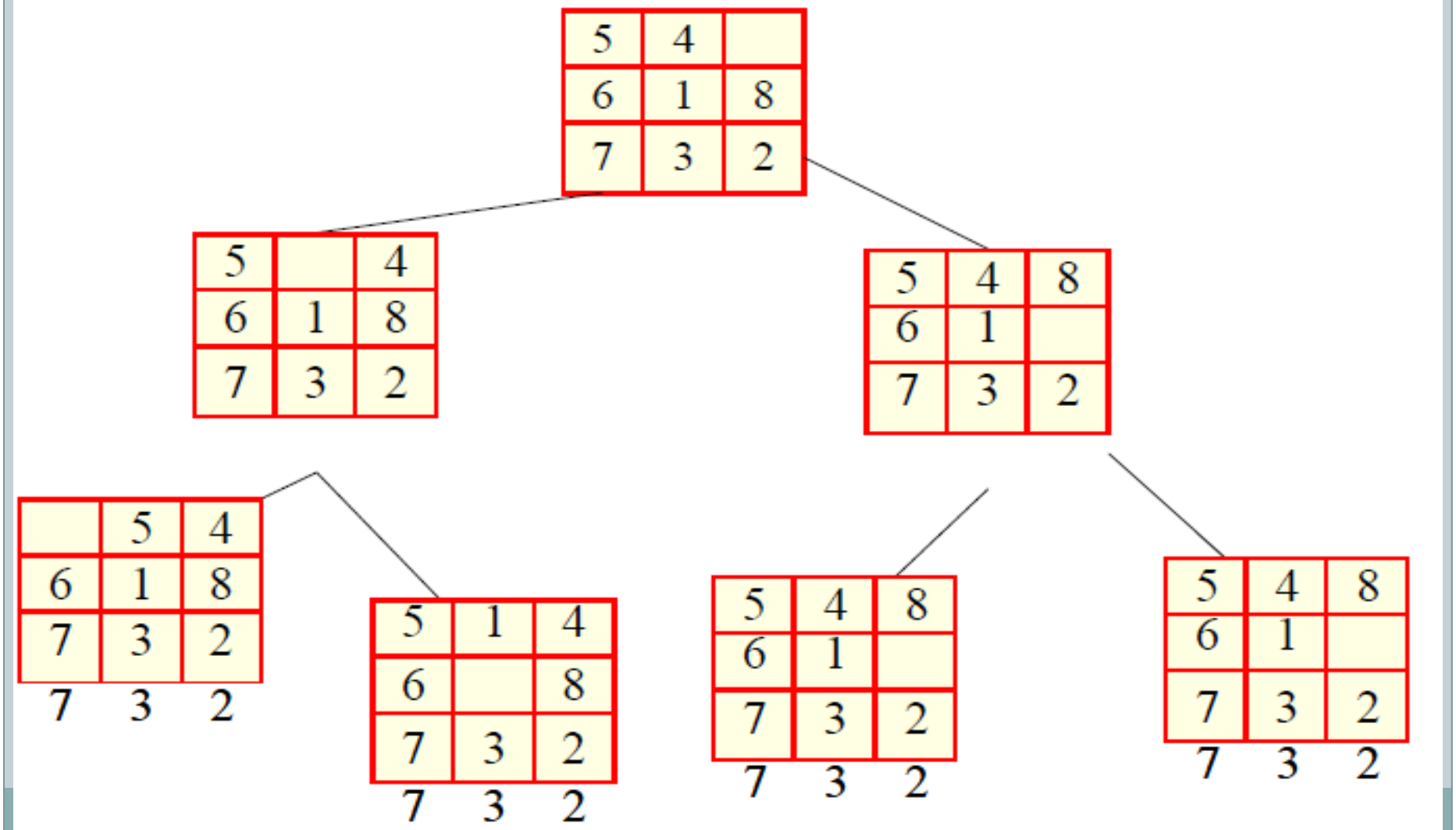


Figure 22

In the 8-puzzle problem we have a 3 x 3 square board and 8 numbered tiles. The board has one blank position. Bocks can be slid to adjacent blank positions. We can alternatively and equivalently look upon this as the movement of the blank position up, down, left or right. The objective of this puzzle is to move the tiles starting from an initial position and arrive at a given goal configuration.

Example: 8 puzzle



Example: 8 puzzle

- **The problem formulation is as follows:**
- **States:** It describes the location of each numbered tiles and the blank tile.
- **Initial State:** We can start from any state as the initial state.
- **Actions:** Here, actions of the blank space is defined, i.e., either **left, right, up or down**
- **Transition Model:** It returns the resulting state as per the given state and actions.
- **Goal test:** It identifies whether we have reached the correct goal-state.
- **Path cost:** The path cost is the number of steps in the path where the cost of each step is 1.

Search Algorithm Technologies

Problem-solving agents in AI mostly used these **search strategies** or algorithms to solve a specific problem and provide the best result.

Search: Searching is a step by step procedure to solve a search-problem in a given search space.

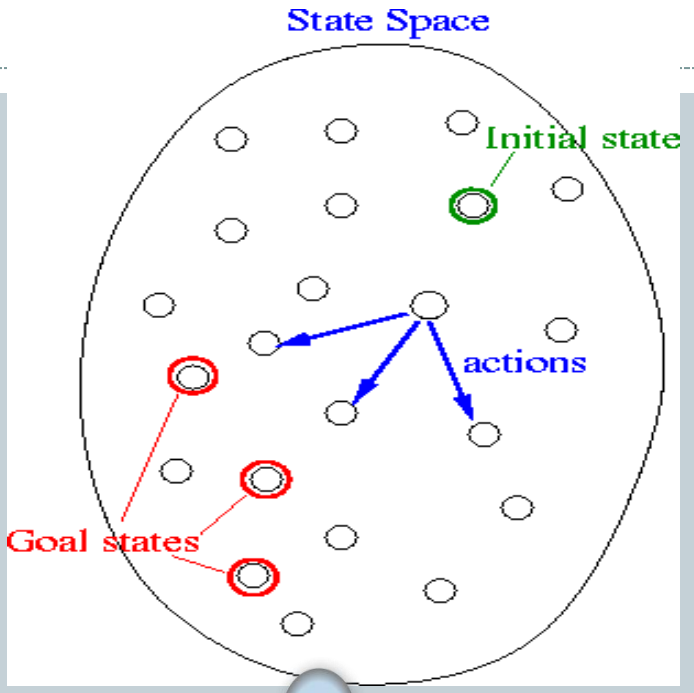
A search problem can have three main factors:

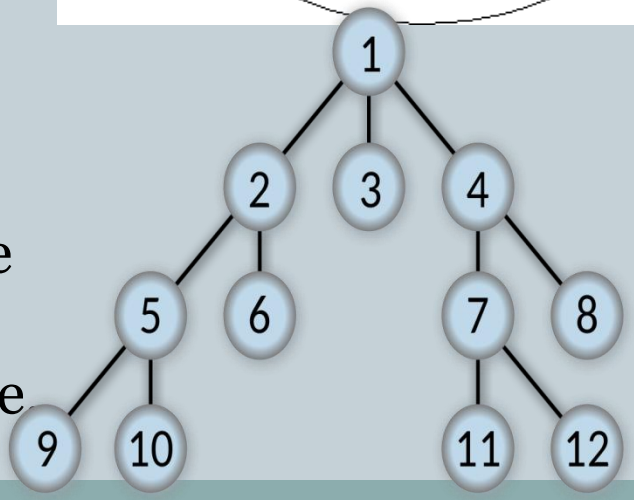
- **Search Space:** Search space represents a **set of possible solutions**, which a system may have.
- **Start State:** It is a state from where agent begins **the search**.
- **Goal test:** It is a function which observe the current state and returns whether the goal state is achieved or not.

Search Algorithm Technologies

- **State Space:** A **state space** represents a problem in terms of **states** and **operators** that change **states**
- **Start State:** It is a state from where agent begins **the search**.
- **Goal test:** It is a function which observe the current state and returns whether the goal state is achieved or not.

Search tree: A tree representation of search problem is called Search tree. The root of the search tree is the root node which is corresponding to the initial state.



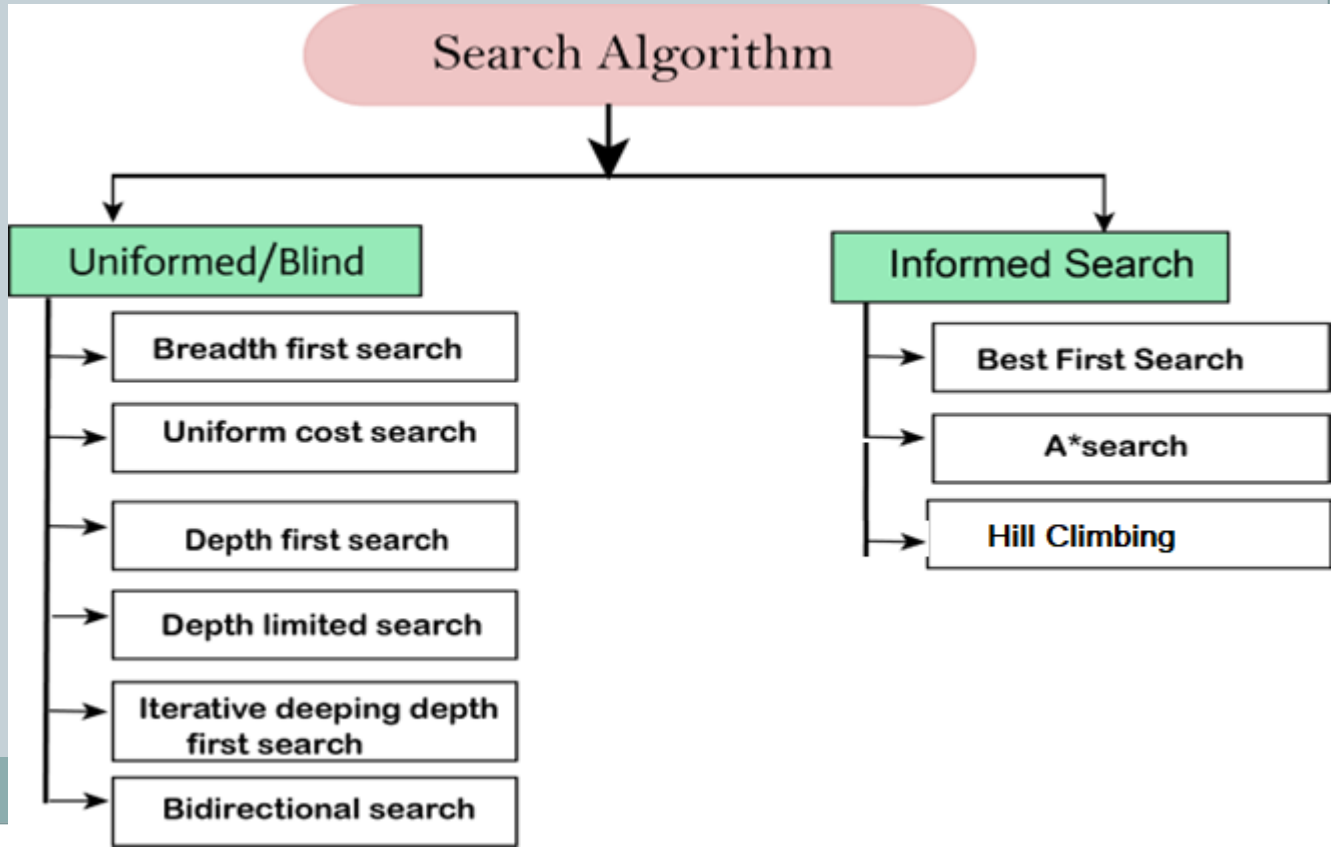


Search Algorithm Technologies

- **Actions:** It gives the description of all the available actions to the agent.
- **Transition model:** A description of what each action do, can be represented as a transition model.
- **Path Cost:** It is a function which assigns a numeric cost to each path.
- **Solution:** It is an action sequence which leads from the start node to the goal node.
- **Optimal Solution:** If a solution has the lowest cost among all solutions.

Types of search algorithms

- Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



Problem Space

- The problem space may contain one or more solutions.
- A solution is a **combination** of operations and objects that achieve the goals.

- A problem space **encompasses** all valid states that can be generated by the **application** of any **combination** of operators on any combination of objects.

Problem solving is a process of generating solutions from observed data.

A 'problem' is characterized by

- a set of **goals**,
- a set of **objects**,
- a set of **operations**

Uninformed/Blind Search

- The uninformed search does not contain any **domain knowledge** such as closeness, the location of the goal.
- It operates in a **brute-force** way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes.
- Uninformed search applies a way in which search tree is searched **without** any **information** about the search space like initial state operators and test for the goal, so it is also called **blind search**.
- It examines each **node** of the tree until it achieves the **goal** node.

Informed Search

- **The Informed Search** (or Heuristic Search) is searching with "information" about the goal node.
- For example, in the A* search algorithm, first, we gather information about the goal node such as its location in cartesian coordinate form and then we write a function that can guide our node traversal, for example, finding its Euclidean distance, also this function is called Heuristic Function.
- The heuristic function will **guide** our **node traversal** by stating the **distance** left from each node to the goal node.
- You can also think of Informed Search Strategy as a Guided Search Strategy.

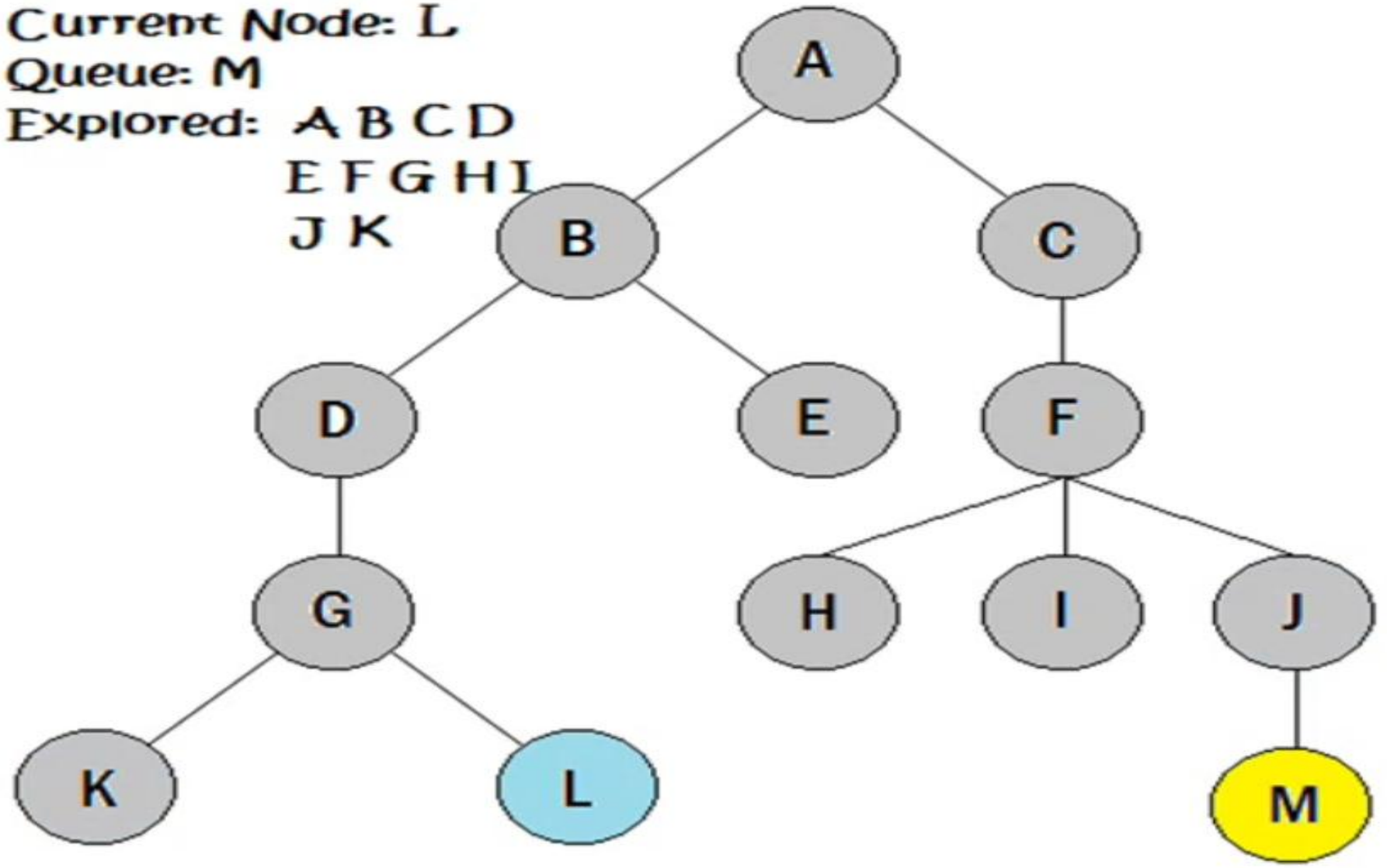
Informed Search

- Informed search algorithms use domain knowledge. In an informed search, problem information is available which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. Informed search is also called a Heuristic search.
- A heuristic is a way which might not always be guaranteed for best solutions but guaranteed to find a good solution in reasonable time.
- Informed search can solve much complex problem which could not be solved in another way.
- An example of informed search algorithms is a traveling salesman problem.
  - Greedy Search
  - A* Search

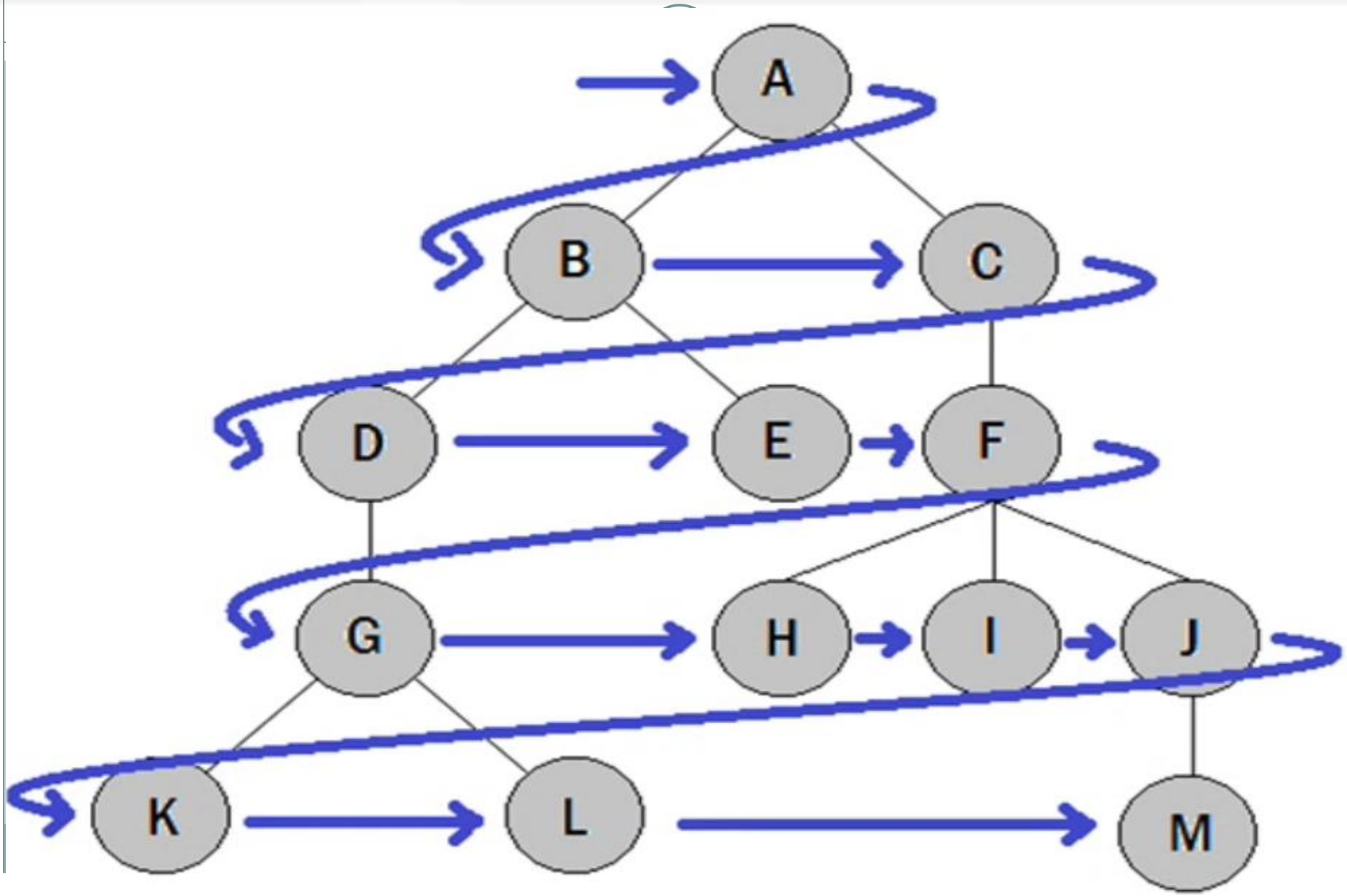
Blind Search: Breadth-first Search

- Breadth-first search is the most common search strategy for traversing a tree or graph.
- This algorithm searches **breadthwise** in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the **root node** of the tree and **expands** all successor node at the **current level** before moving to nodes of next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first search implemented using FIFO (First-In-First-Out) queue data structure.

Breadth-first Search



Breadth-first Search



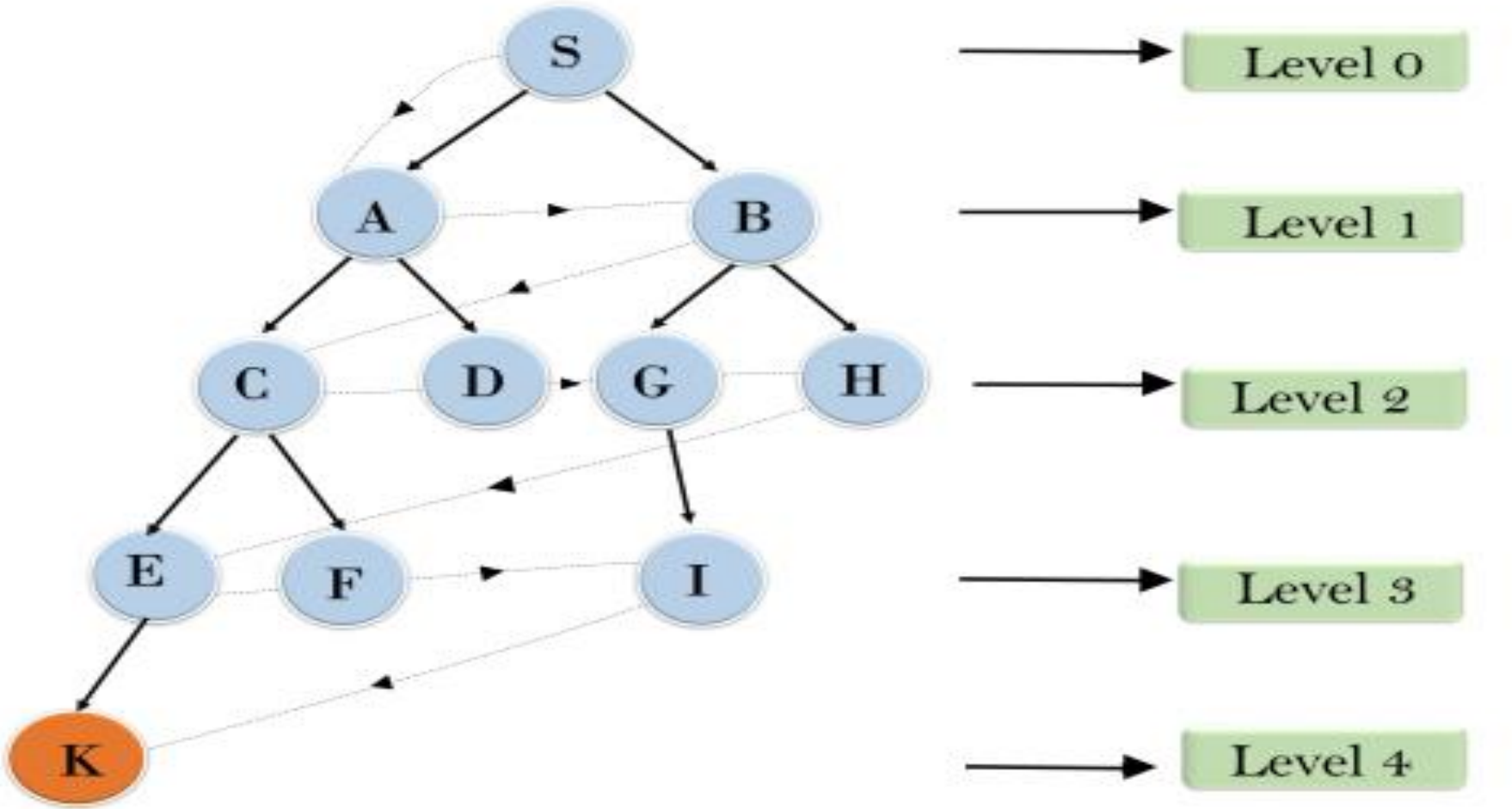
Breadth-first Search

Example:

- In the below tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K.
- BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

S---> A-->B--->C-->D--->G-->H-->E-->F-->I---->K

Breadth First Search



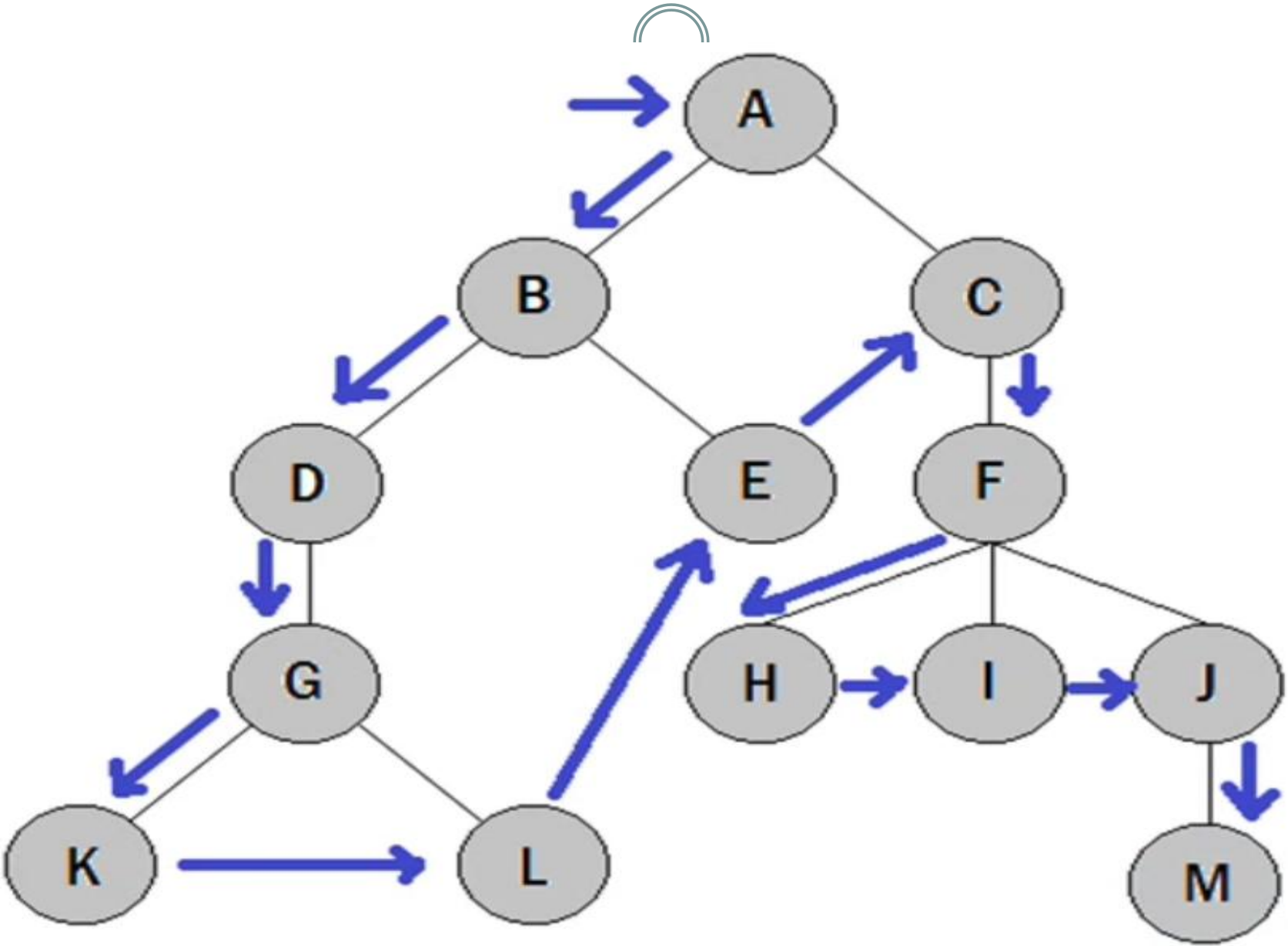
S---> A-->B--->C-->D--->G-->H-->E-->F-->I---->K

Depth-first Search

- Depth-first search is a **recursive** algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the **root node** and follows each path to its **greatest** depth node before moving to the next path.
- DFS uses a **stack data** structure for its implementation.
- The process of the DFS algorithm is similar to the BFS algorithm.

Stack: Consider an example of plates stacked over one another in the canteen. The plate which is at the top is the first one to be removed, i.e. the plate which has been placed at the bottommost position remains in the stack for the longest period of time. So, it can be simply seen to follow LIFO(Last In First Out)/ FILO(First In Last Out) order.

Depth-first Search



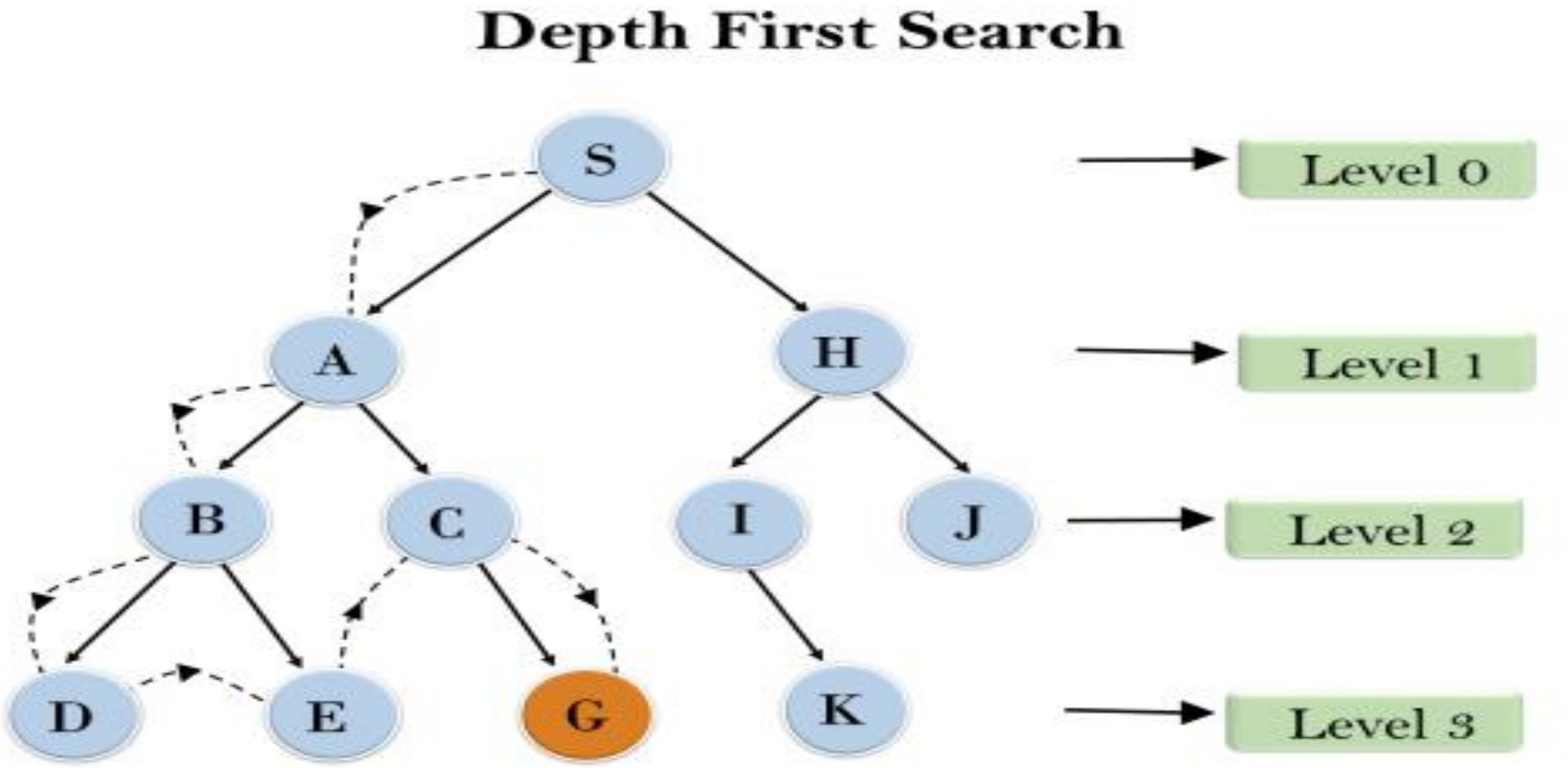
Depth-first Search: Example

- In the below search tree, we have shown the flow of depth-first search, and it will follow the order as:

  Root node--->Left node ----> right node.

- It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found.
- After backtracking it will traverse **node C** and then **G**, and here it will terminate as it found goal node.

Depth-first Search: Example



Depth-first with Iterative Deepening

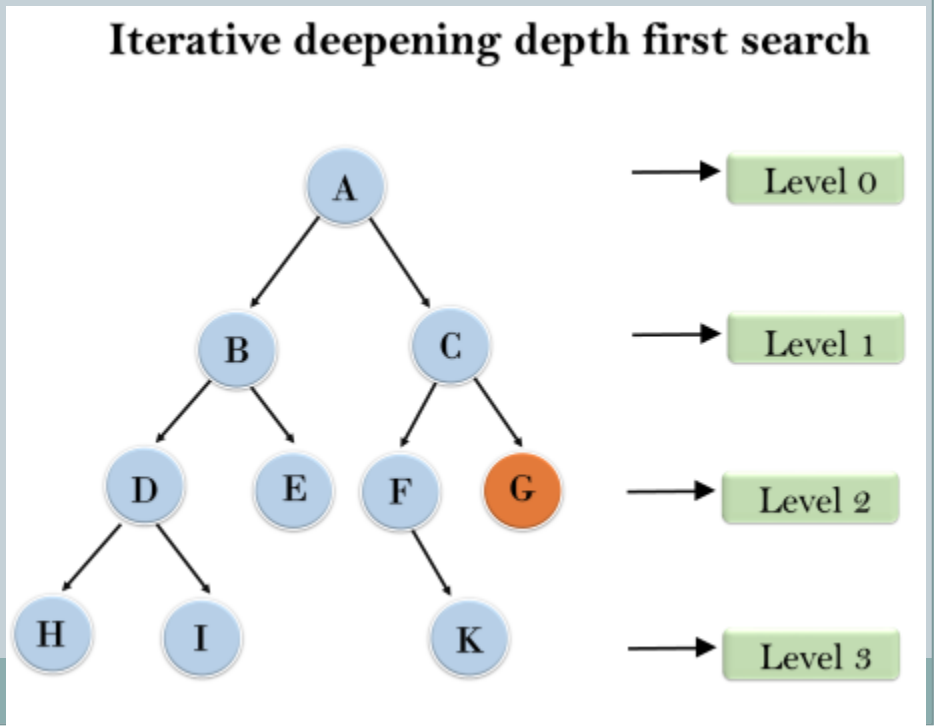
- The iterative deepening algorithm is a **combination** of **BFS** and **DFS** algorithms.
- This search algorithm finds out the **best depth limit** and does it by **gradually increasing** the limit until a **goal** is **found**.
- This algorithm performs **depth-first** search up to a certain "depth **limit**", and it keeps **increasing** the depth limit after each iteration until the goal node is found.
- This Search algorithm combines the benefits of **Breadth-first** search's fast search and depth-first search's memory efficiency.
- The iterative search algorithm is useful uninformed search when search space is large, and depth of goal node is unknown.

Depth-first with Iterative Deepening (DFIDS): Example

- Following tree structure is showing the iterative deepening depth-first search **(DFIDS)**.
- **DFIDS** algorithm performs various iterations until it does not find the goal node. The iteration performed by the algorithm is given as:

1'st Iteration-----> A
2'nd Iteration----> A, B, C
3'rd Iteration------>A, B, D, E, C, F, G
4'th Iteration------>A, B, D, H, I, E, C, F, K, G

In the fourth iteration, the algorithm will find the goal node.



COMPARING TYPES OF SEARCH STRATEGY

- Also known as Heuristic Seach.
- Requires Information to perform search.

- Also known as Blind Search.
- Do not require information to perform search.

- Accuracy trade off for speed & time.
- Good solution accepted as optimum solution.

- Speed & time trade off for accuracy.
- Best solution can be achieved.

- Less computational requirement.
- Can handle large search problem.
- Less costly and more efficient.

- More computational requirement.
- Impractical to solve large search problem.
- More costly and less effecient.

Example
- A* Algorithm
- Best First Search
- Hill Climbing Algo
- Beam Search
- AO* Algorithm

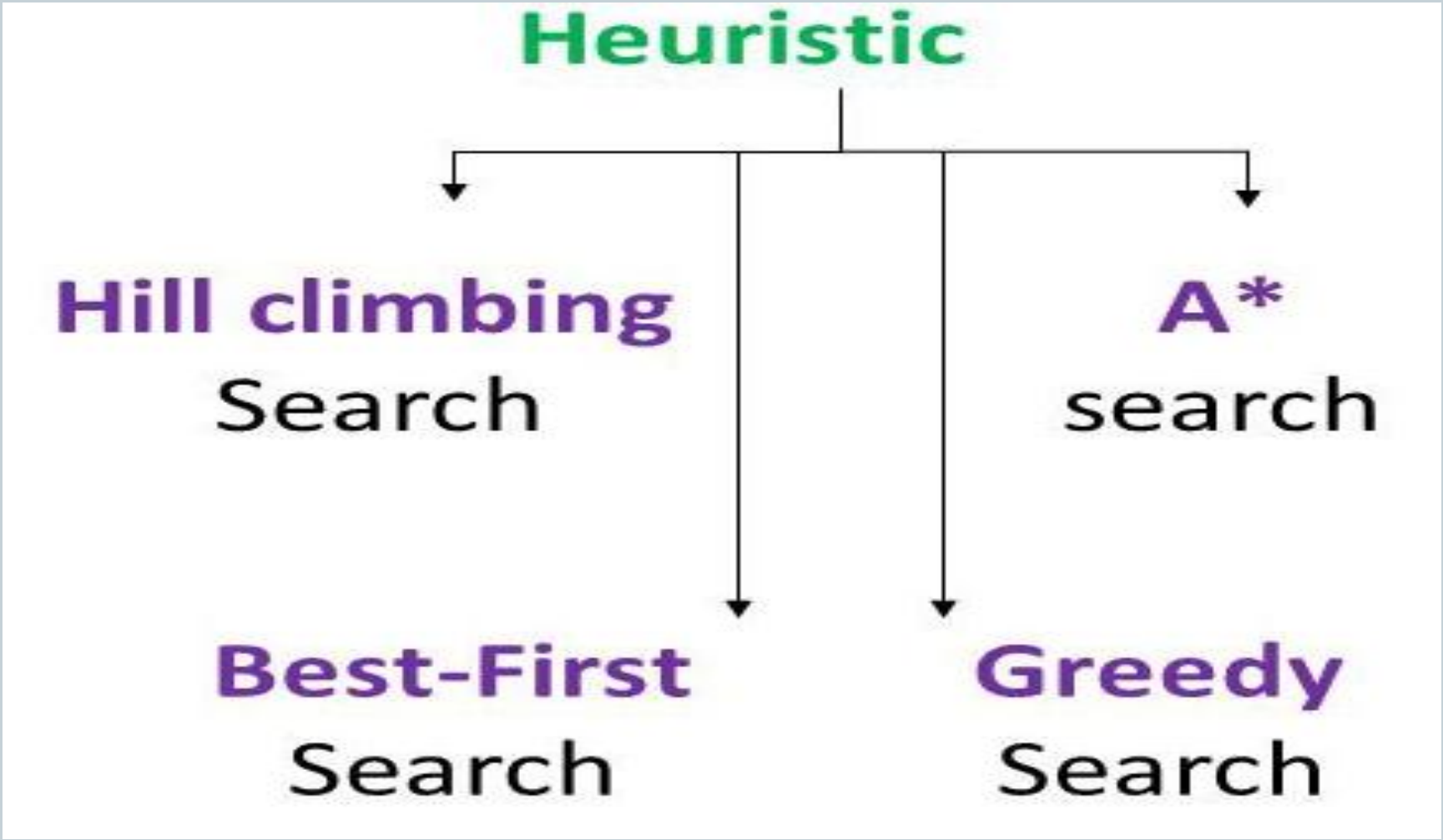
Example
- Breadth-First Search
- Depth-First Search
- Uniform Cost Search
- Bidirectional Search
- Branch and Bound

Informed / Heuristic Search

- Informed search algorithm contains an **array of knowledge** such as how **far** we are from the **goal**, **path cost**, how to **reach** to goal node, etc.
- This knowledge help agents to explore less to the **search space** and find more **efficiently** the goal node.
- The informed search algorithm is more useful for **large search space**.
- Informed search algorithm uses the idea of heuristic, so it is also called Heuristic search.

Heuristic: Find or Discover

Informed / Heuristic Search



Hill Climbing

- Hill Climbing is a form of heuristic search algorithm which is used in solving **optimization** related problems in AI domain.

Searching for a goal state = Climbing to the top of a hill

- **Generate-and-test + direction to move.**
- Heuristic function to estimate how close a given state is to a goal state.

Generate-and-Test Algorithm

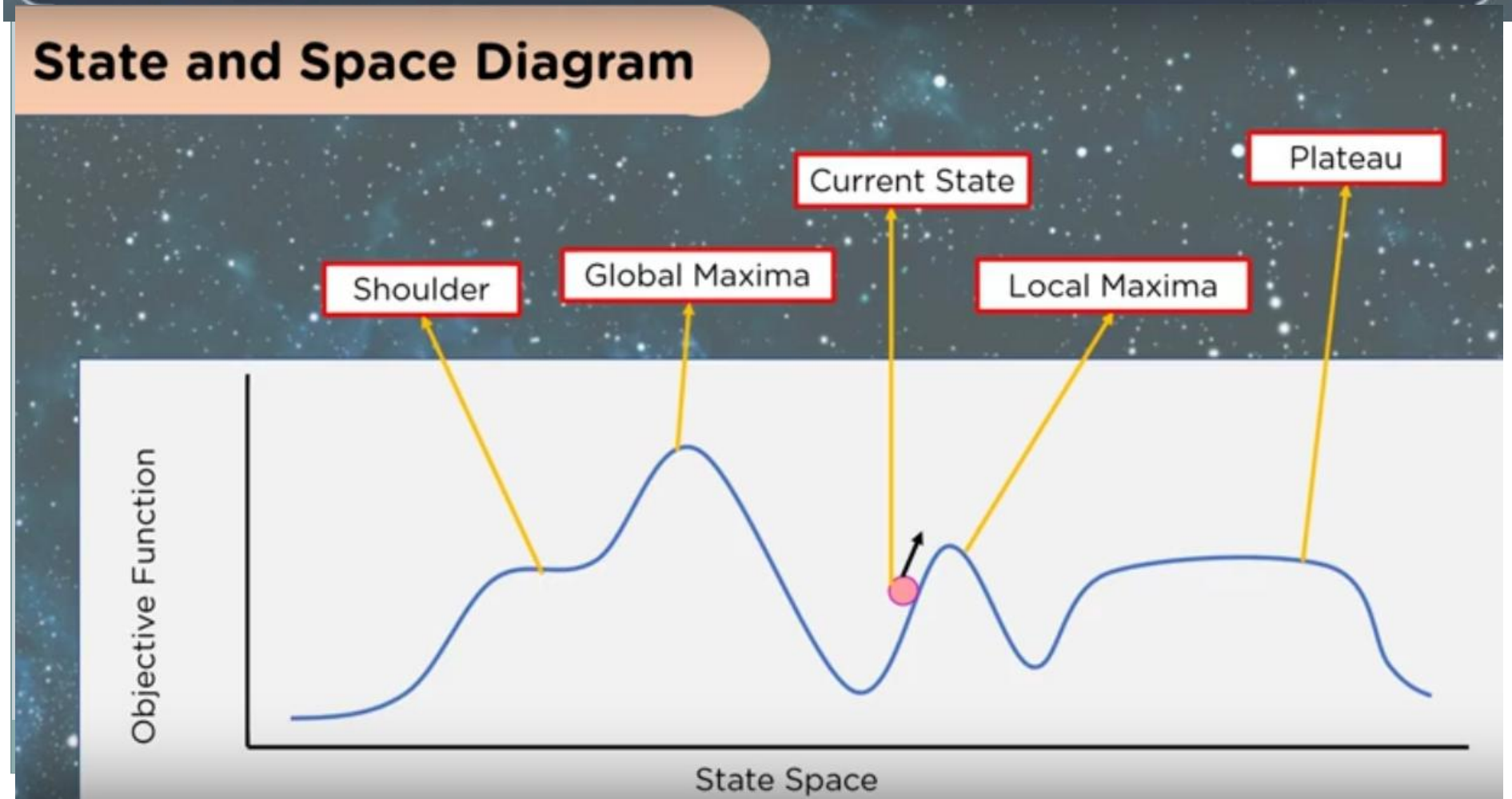
1. Generate a possible solution.
2. Test to see if this is actually a solution.
3. Quit if a solution has been found. Otherwise, return to step 1.

Hill Climbing

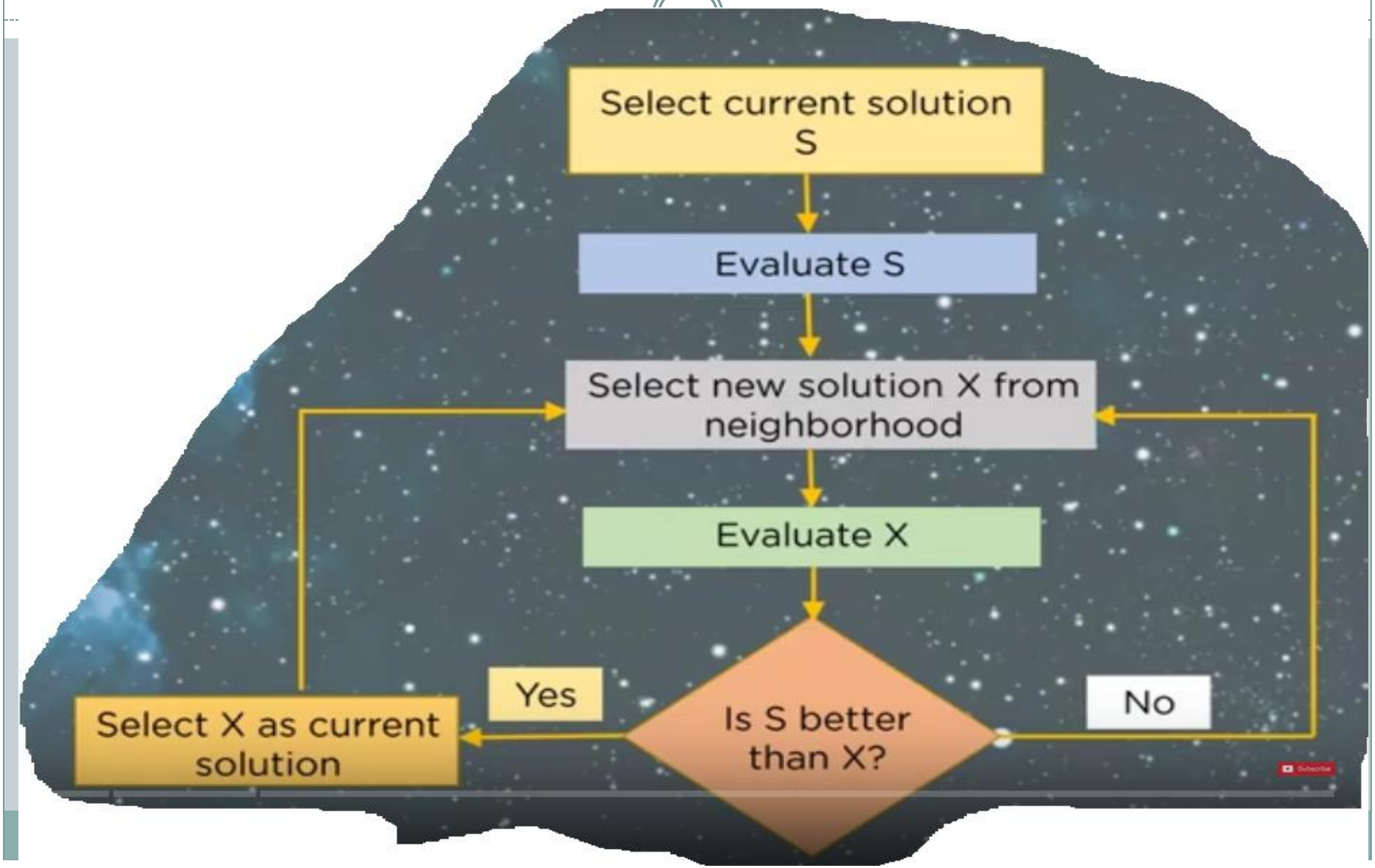
- The algorithm starts with a **non-optimal** state and **iteratively** improves its state **until** some predefined **condition** is met.
- The condition to be met is based on the **heuristic function**. *(Heuristic function **estimates** how **close** a state is to the **goal**).*
- The aim of the algorithm is to reach an **optimal state** which is **better** than its **current** state.
- The starting point which is the non-optimal state is referred to as the **base** of the hill and it tries to constantly iterate (**climb**) untill it reaches the peak value, that is why it is called Hill Climbing Algorithm

Hill Climbing

Hill Climbing Algorithm is a local search algorithm which is used to find the peak of a mountain or the best solution to a problem by continuously moving in the direction of increasing elevation



Hill Climbing



Hill Climbing

- Hill climbing algorithm is a **local search** algorithm which **continuously moves** in the direction of increasing **elevation**/value to find the **peak** of the mountain or best solution to the problem.
- It **terminates** when it reaches a **peak** value where no neighbor has a higher value.
- Hill climbing algorithm is a technique which is used for **optimizing** the **mathematical** problems.
- One of the widely discussed examples of Hill climbing algorithm is **Traveling-salesman** Problem in which we need to minimize the distance traveled by the salesman.

Hill Climbing

- It is also called **greedy local** search as it only looks to its good **immediate** neighbor state and not beyond that.
- A node of hill climbing algorithm has two components which are **state** and **value**.
- In this algorithm, we **don't** need to maintain and handle the **search tree** or graph as it only keeps a single current state.

HC: Different regions in the state space landscape

- **Local Maximum:** Local maximum is a state which is better than its **neighbor states**, but there is also another state which is higher than it.
- **Global Maximum:** Global maximum is the best possible state of state **space landscape**. It has the highest value of objective function.
- **Current state:** It is a state in a landscape diagram where an agent is currently present.
- **Flat local maximum:** It is a flat space in the landscape where all the neighbor states of current states have the **same value**.
- **Shoulder:** It is a **plateau** region which has an uphill edge.

Features of Hill Climbing

Following are some main features of Hill Climbing Algorithm:

- **Generate and Test variant:** Hill Climbing is the variant of Generate and Test method. The Generate and Test method produce **feedback** which helps to **decide** which **direction** to move in the search space.
- **Greedy approach:** Hill-climbing algorithm search moves in the **direction** which **optimizes** the cost.
- **No backtracking:** It does not backtrack the search space, as it does not remember the previous states.

Heuristics Function ...h(n)

- Heuristic is a function which is used in Informed Search, and it finds the **most promising path**.
- It takes the **current state** of the agent as its input and produces the estimation of how close agent is from the goal.
- The heuristic method, however, might not always give the best solution, but it **guaranteed** to find a **good solution** in reasonable **time**.
- Heuristic function **estimates** how **close** a state is to the **goal**. It is represented by **h(n)**, and it calculates the cost of an optimal path between the pair of states.
- The value of the heuristic function is always positive.

$$h(n) <= h^*(n)$$

Here h(n) is heuristic cost, and h*(n) is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

Best-First Search

- Greedy best-first search algorithm always selects the **path** which appears **best** at that **moment**.
- It is the **combination** of depth-first search and breadth-first search algorithms.
- It uses the heuristic **function** and **search**.
- Best-first search allows us to take the advantages of both algorithms.
- With the help of best-first search, at each step, we can choose the most promising node.

Best-First Search

- In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e.

f(n)= h(n)

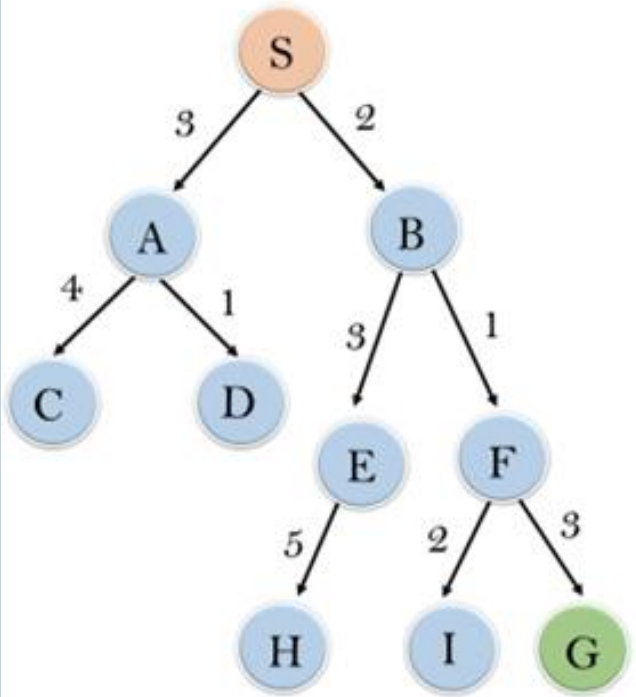
Where, h(n)= estimated cost from node n to the goal.

Generic Best-First Search

- **Step 1:** Place the starting node into the OPEN list.
- **Step 2:** If the OPEN list is empty, Stop and return failure.
- **Step 3:** Remove the node n, from the OPEN list which has the lowest value of h(n), and places it in the CLOSED list.
- **Step 4:** Expand the node n, and generate the successors of node n.
- **Step 5:** Check each successor of node n, and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- **Step 6:** For each successor node, algorithm checks for evaluation function f(n), and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- **Step 7:** Return to Step 2.

Generic Best-First Search

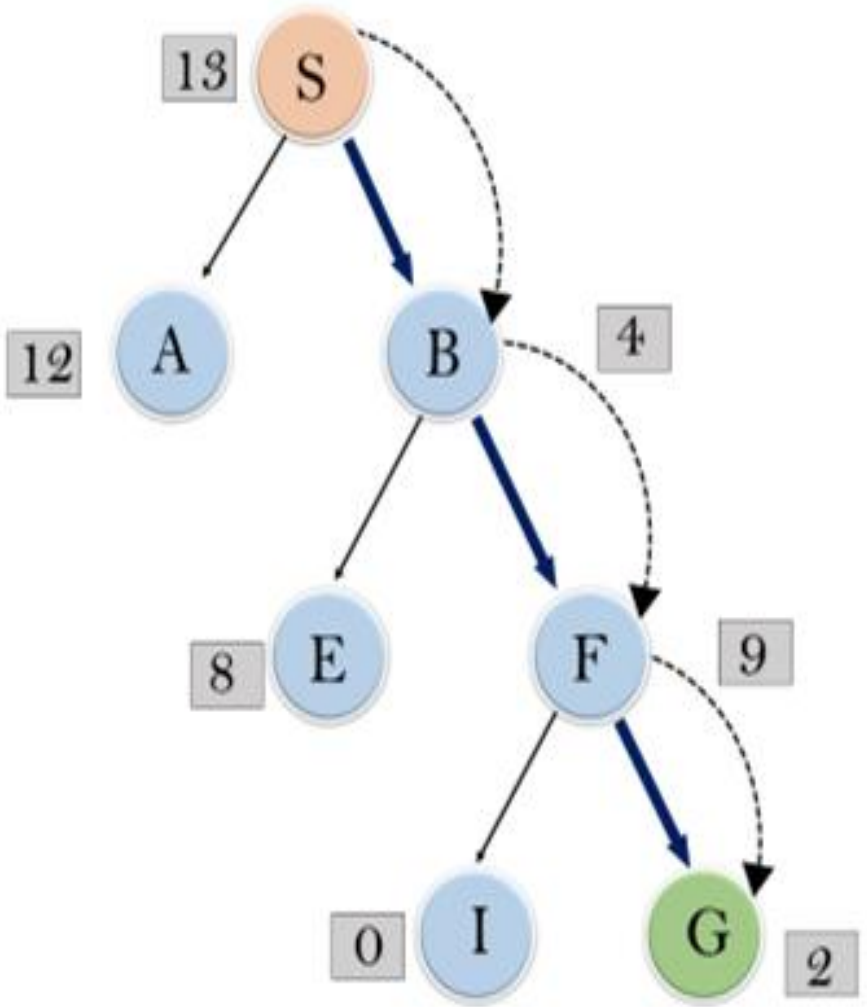
- Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function f(n)=h(n) , which is given in the table.



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

Generic Best-First Search

- In this search example, we are using two lists which are **OPEN** and **CLOSED** Lists.
- Following are the iteration for traversing the above example.



Generic Best-First Search

- **Expand the nodes of S and put in the CLOSED list**
- **Initialization:** Open [A, B], Closed [S]
- **Iteration 1:** Open [A], Closed [S, B]
- **Iteration 2:** Open [E, F, A], Closed [S, B]
  : Open [E, A], Closed [S, B, F]
- **Iteration 3:** Open [I, G, E, A], Closed [S, B, F]
  : Open [I, E, A], Closed [S, B, F, G]
- Hence the final solution path will be:

S----> B----->F----> G

A* Search

- A* search is the most commonly known form of best-first search. It uses heuristic function h(n), and cost to reach the node n from the start state g(n).
- It has combined features of UCS and greedy best-first search, by which it solve the problem efficiently. A* search algorithm finds the shortest path through the search space using the heuristic function.
- This search algorithm expands less search tree and provides optimal result faster. A* algorithm is similar to UCS except that it uses g(n)+h(n) instead of g(n).

A* Search

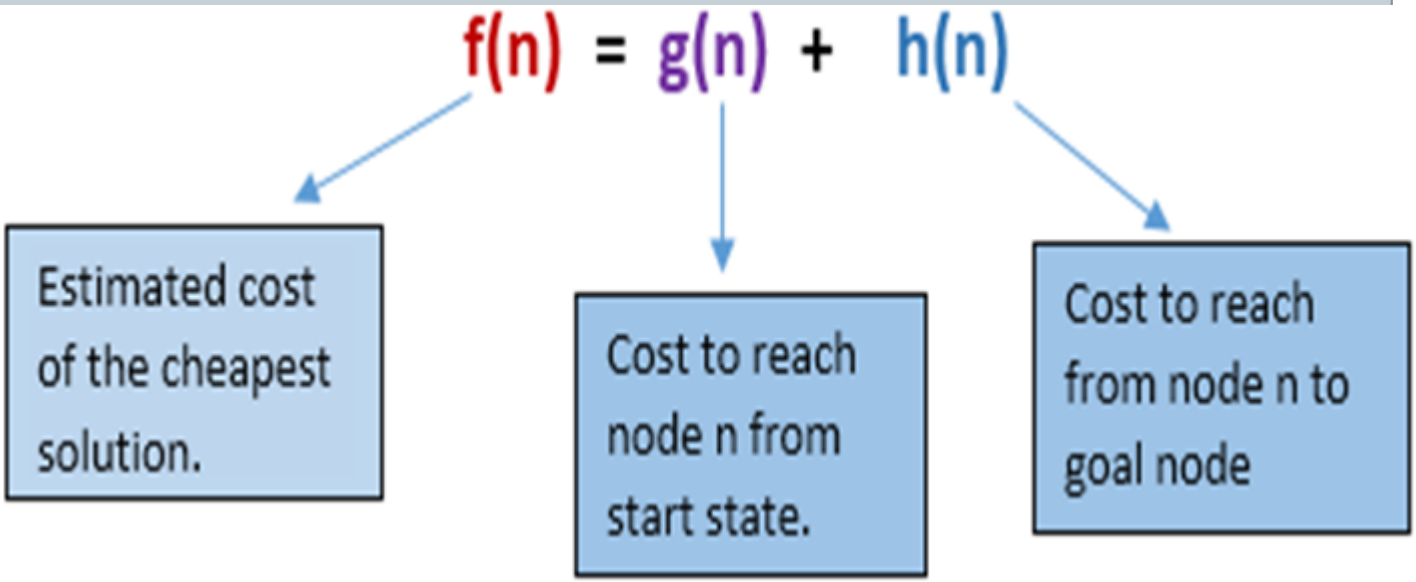
- A* search algorithm finds the **shortest path** through the search space using the heuristic function.
- This search algorithm expands less search tree and provides optimal result faster.

- In A* search algorithm, we use search **heuristic** as well as the **cost** to reach the node.
- Hence we can combine both costs as following, and this sum is called as a **fitness number**.

A* Search

- In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we can combine both costs as following, and this sum is called as a **fitness number [*f(n)*]**.
- A* algorithm uses g(n)+h(n) unlike use of g(n) in other search algorithms.

Used mainly in Games and Web-Based Maps



Algorithm of A* Search

- **Step1:** Place the starting node in the OPEN list.
- **Step 2:** Check if the OPEN list is empty or not, if the list is empty then return failure and stops.
- **Step 3:** Select the node from the OPEN list which has the smallest value of evaluation function (g+h), if node n is goal node then return success and stop, otherwise
- **Step 4:** Expand node n and generate all of its successors, and put n into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.
- **Step 5:** Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest g(n') value.
- **Step 6:** Return to **Step 2**.

A* Search: Example

- In this example, we will traverse the given graph using the A* algorithm. The heuristic value of all states is given in the below table so we will calculate the f(n) of each state using the formula of fitness number

  f(n)= g(n) + h(n)

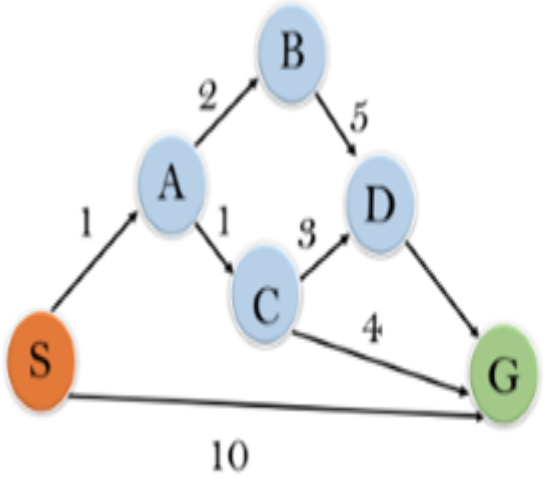
  where g(n) is the cost to reach any node from start state

  h(n) is the cost to reach from node n to goal node
- Here we will use OPEN and CLOSED list.

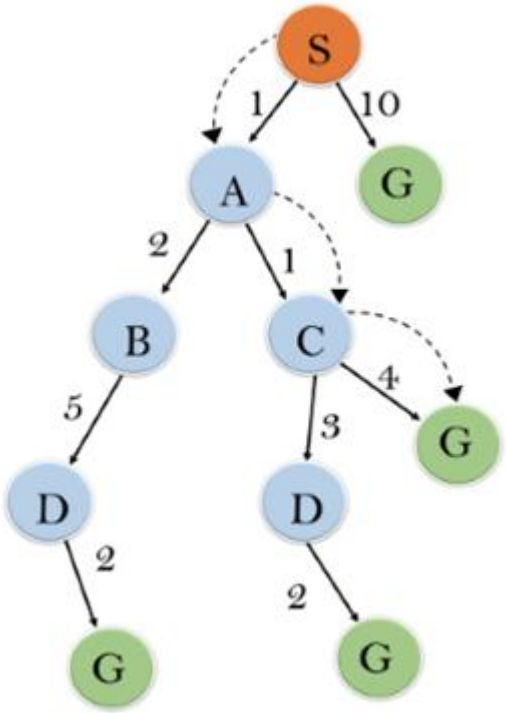
A* Search: Example

use **f(n)= g(n) + h(n)**

- **Initialization:** {(S, 5)}
  S => 0 + 5 = 5
- **Iteration1:**
  S--> A => 1+3= 4
  S-->G => 10+0= 10
- **Iteration2:**
  S--> A-->C =>
  1+1+2 = 4
  S--> A-->B =>
  1+2+4 = 7



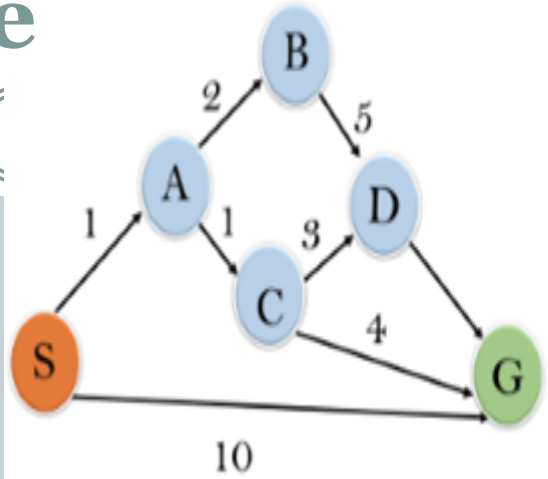
State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0



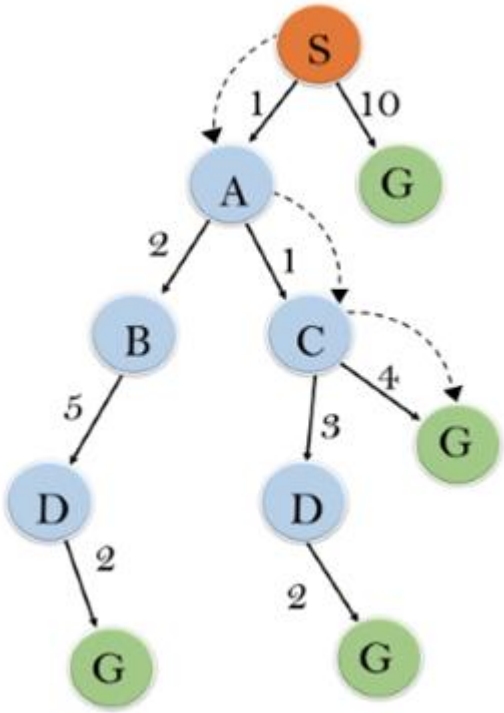
A* Search: Example

- **Iteration3:**
  S--> A-->C--->G =>
  1+1+4+0 =6
  S--> A-->C--->D =>
  1+1+3+6= 11

- **Iteration 4** will give the final result, as **S--->A--->C--->G** it provides the optimal path with cost 6.



State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0



References

- https://www.javatpoint.com/artificial-intelligence-tutorial
- https://www.youtube.com/watch?v=PzEWHH2v3TE
- **Artificial Intelligence: A Modern Approach**
  by **Stuart Russell and Peter Norvig**
- http://aima.cs.berkeley.edu/figures.pdf

END